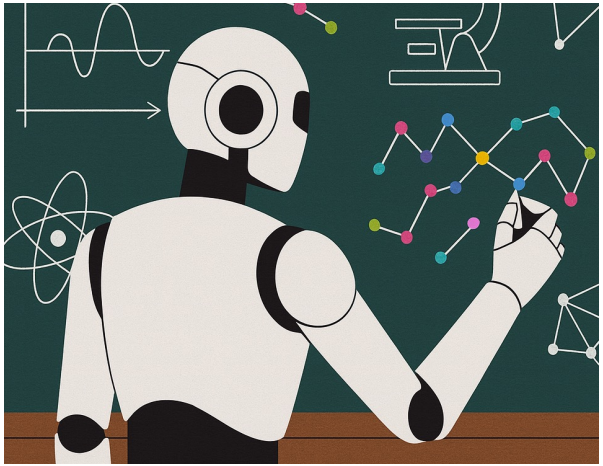
If you have any questions, feel free to reach out!
xuemei.gu@uni-jena.de

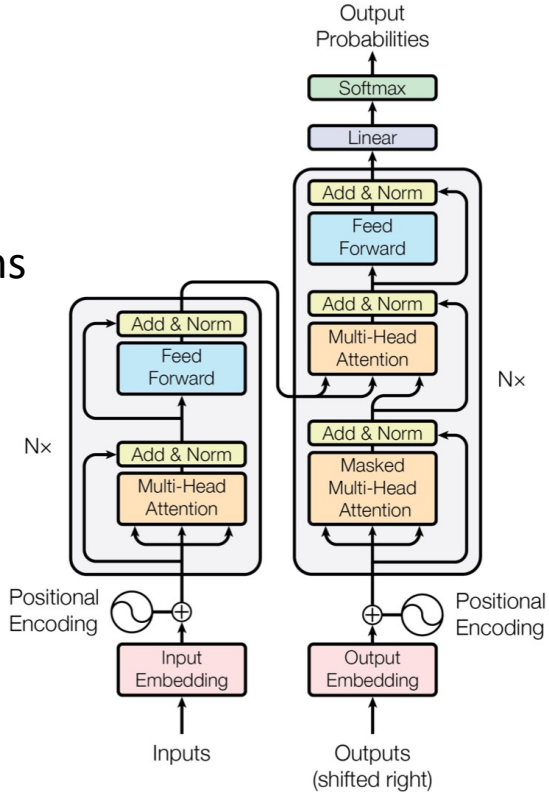
AI for Quantum Physics



Winter semester 2025/26

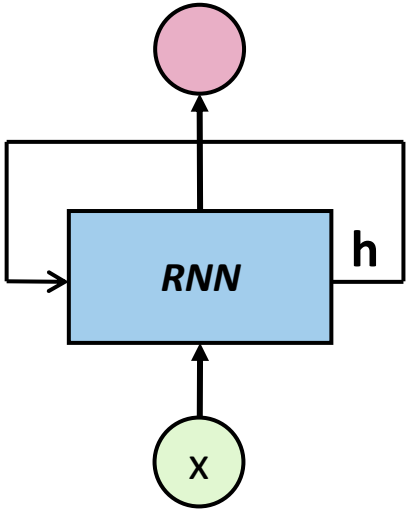
Today's topic:

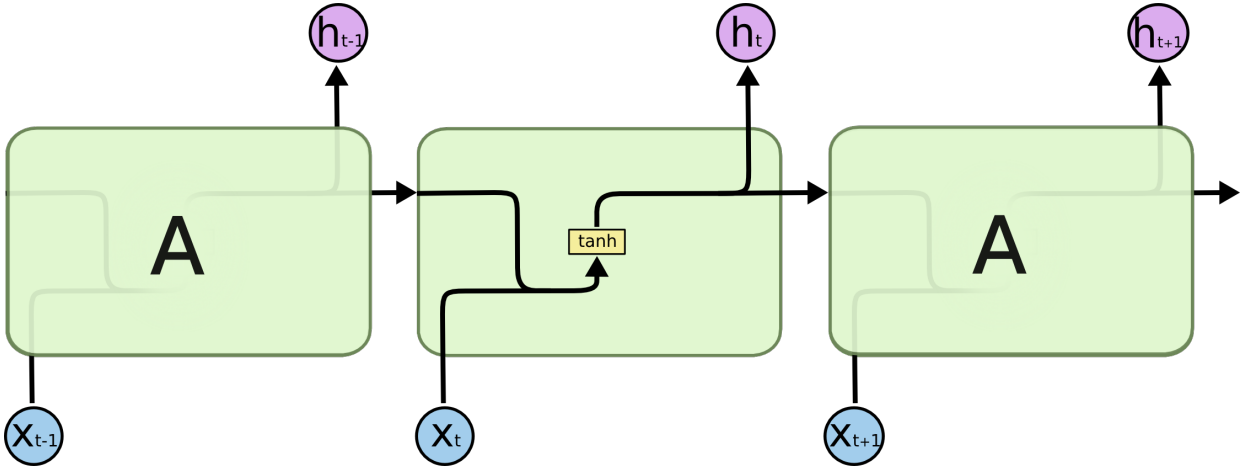
- Attention Mechanisms
- Transformers
- Applications



Recurrent neural network

$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t)$$





State Update

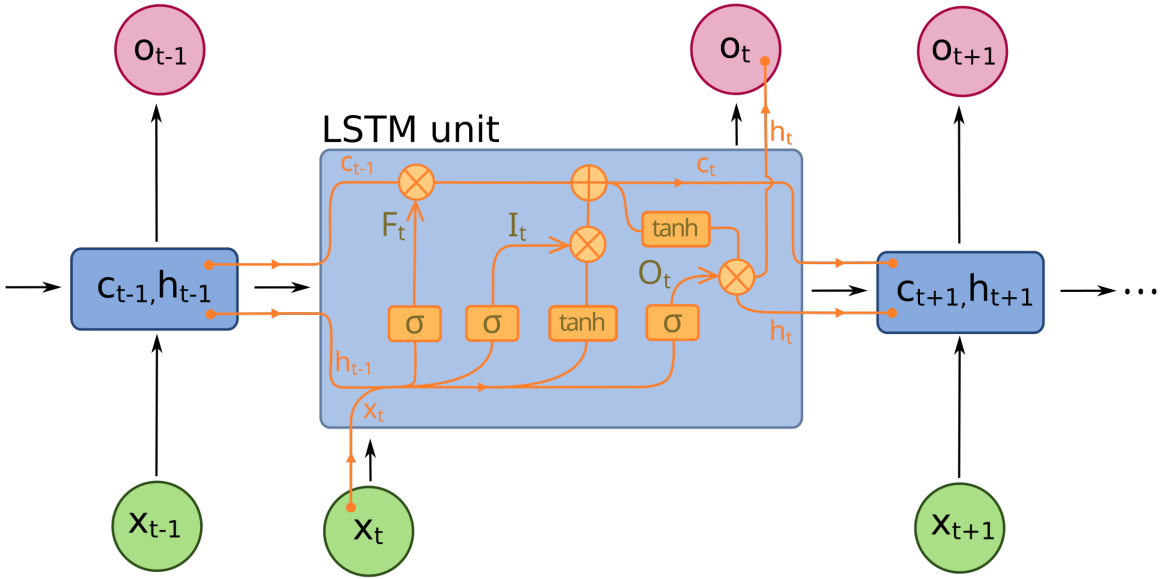
$$h_t = f_W(h_{t-1}, x_t)$$

new state — **old state**

Some function with parameters W

Input vector at some time step

Long short-term memory (LSTM)

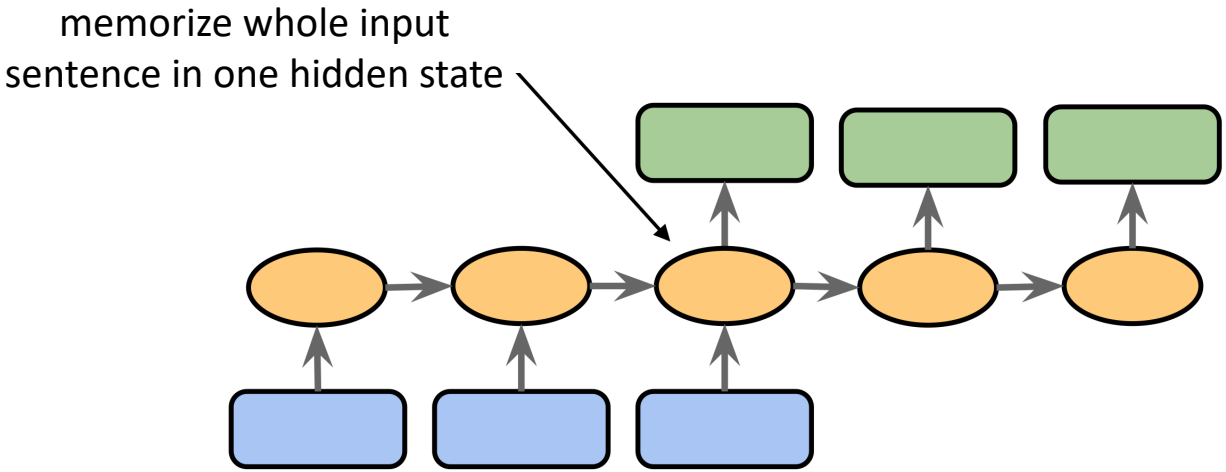


$$
\begin{aligned}
f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
\tilde{c}_t &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \\
h_t &= o_t \odot \sigma_h(c_t)
\end{aligned}
$$


copyright (c) 1999 Daniel J. Simons. All rights reserved.

Attention Mechanism

Originally developed for language translation: Bahdanau, D., Cho, K., & Bengio, Y. (2014).

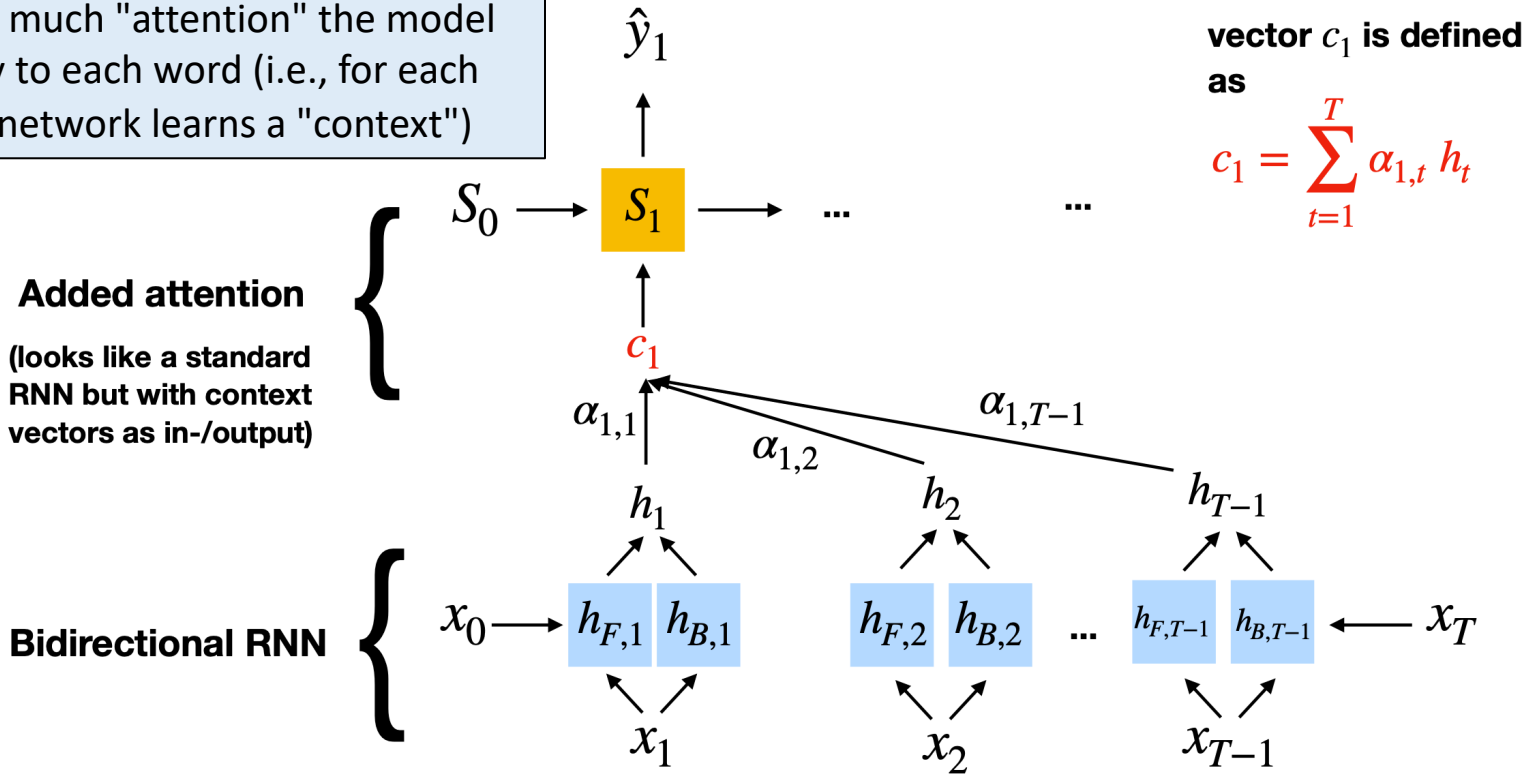


"... allowing a model to automatically (soft-) search for parts of a source sentence that are relevant to predicting a target word ..."

Attention Mechanism

Originally developed for language translation: Bahdanau, D., Cho, K., & Bengio, Y. (2014).

Assign attention weight to each word, to know how much "attention" the model should pay to each word (i.e., for each word, the network learns a "context")



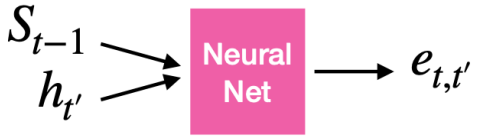
where the context vector c_1 is defined as

$$c_1 = \sum_{t=1}^{T} \alpha_{1,t}\, h_t$$

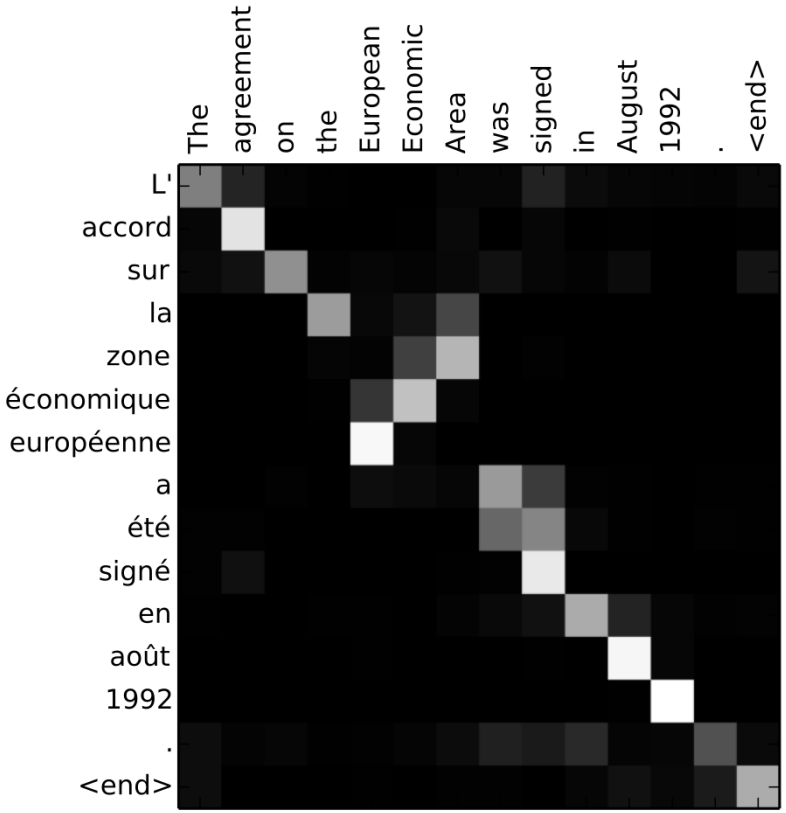
Attention Mechanism

Originally developed for language translation: Bahdanau, D., Cho, K., & Bengio, Y. (2014).

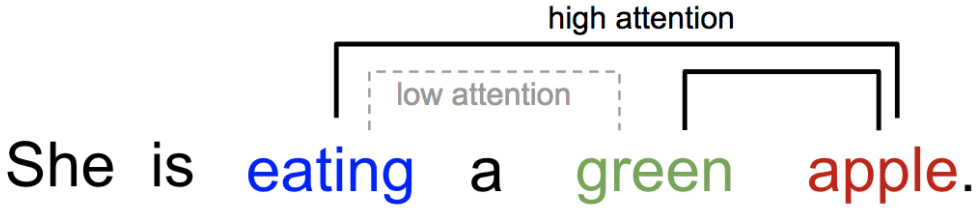
Computing attention weights

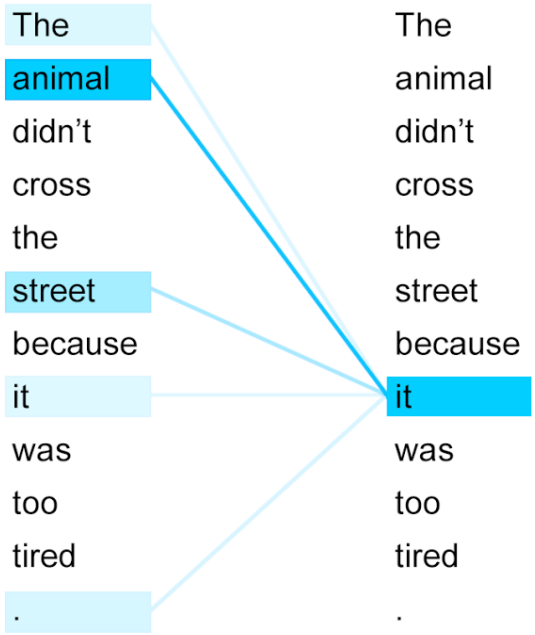


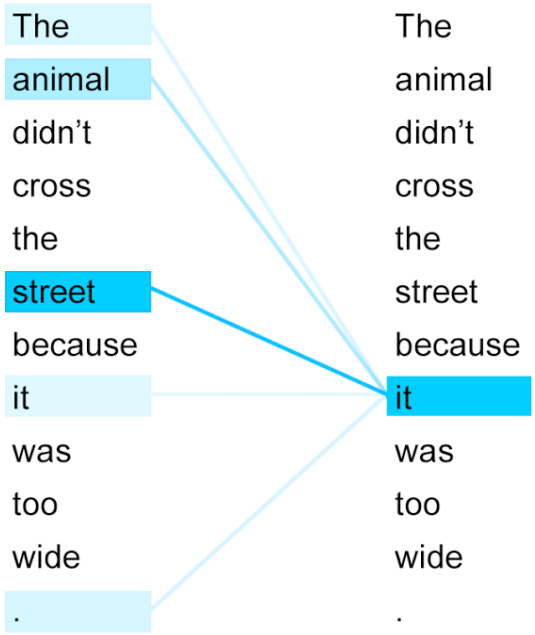
$$\alpha_{t,t'} = \frac{\exp(e_{t,t'})}{\sum_{t'=1}^{T} \exp(e_{t,t'})}$$



Attention Mechanism







Getting rid of the sequential parts ...

Transformer: Self-Attention Mechanism

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

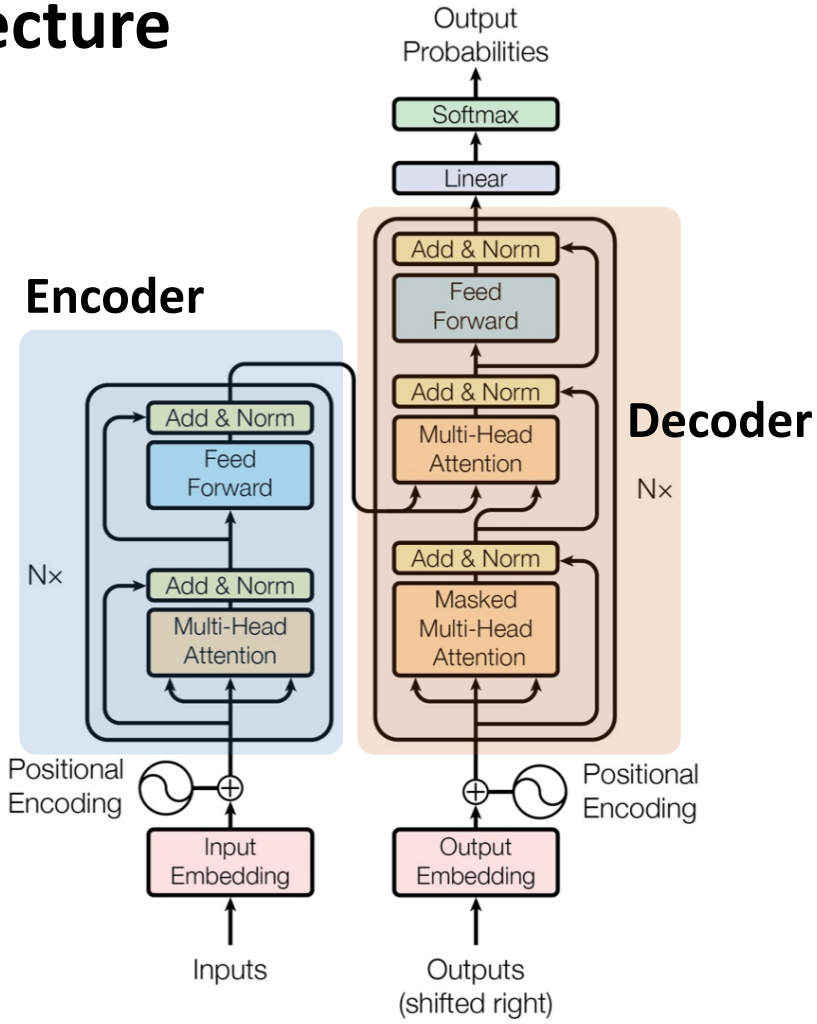
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

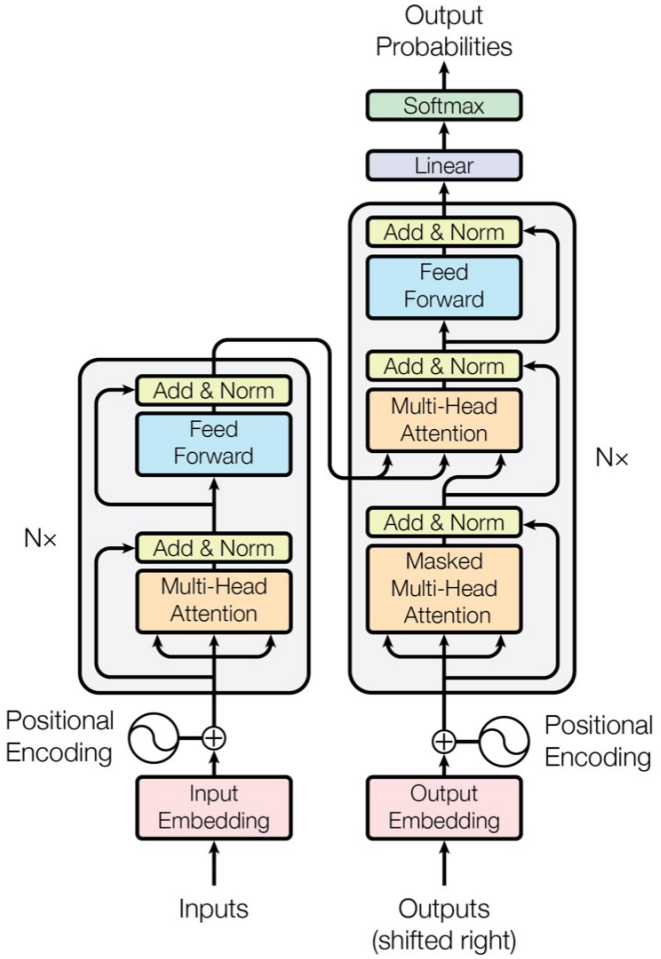


Transformer Architecture





Transformer Architecture



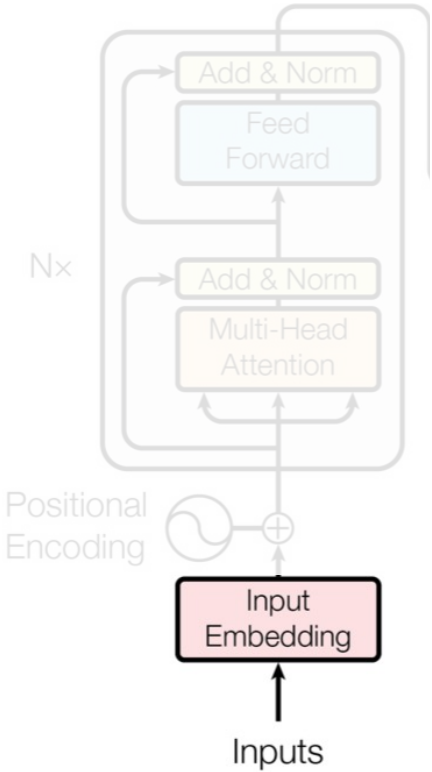
Targets/Outputs

Ich habe einen Apfel gegessen

Inputs

I ate an apple

Processing Inputs



Tokenization the process of converting a sequence of text into smaller parts, known as tokens

GPT-tokenizer playground: https://gpt-tokenizer.dev/

I ate an apple

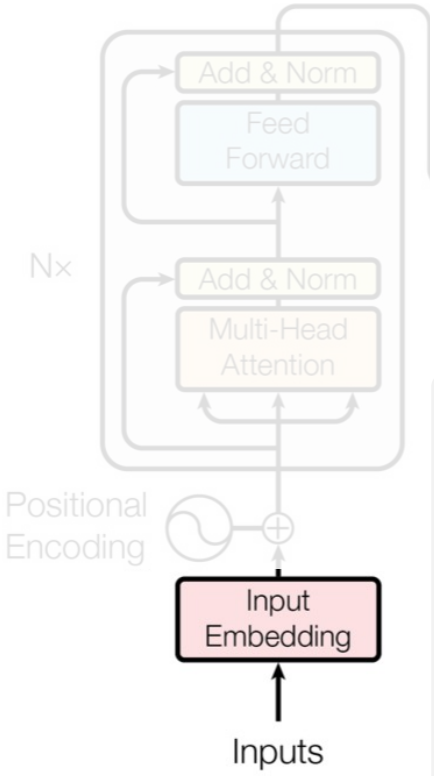
↓

Tokenizer (split into individual words) **Word tokenization**

↓

I | ate | an | apple

Processing Inputs



I ate an apple

↓

Tokenizer (split into individual words)

↓

I ate an apple

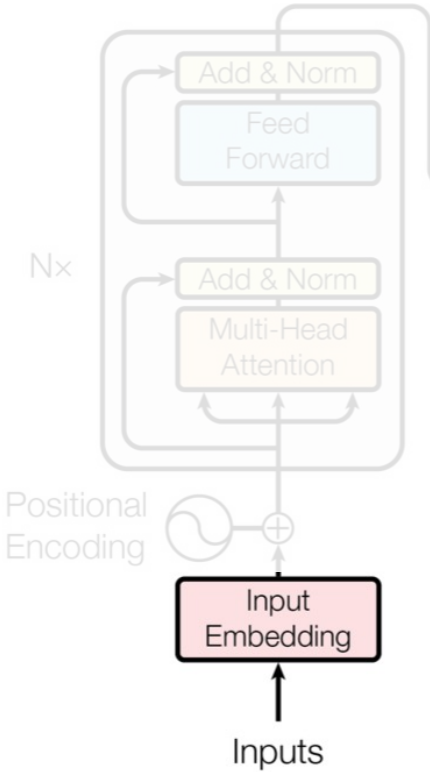
Vocabulary

e.g., (…, nice, the, when, I, …, an, of, …, ate, play, …, apple, …)

indices

(…, 37, 38, 39, 40, …, 448, 449, …, 28397, 28398,…, 30366,…)

Processing Inputs



I ate an apple

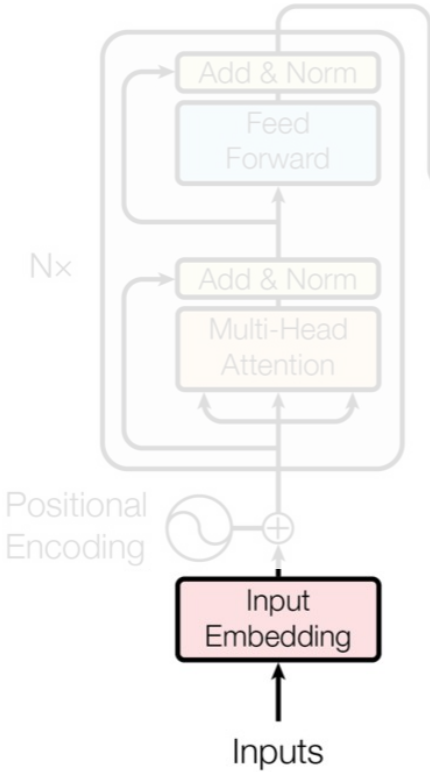
Tokenizer (split into individual words)

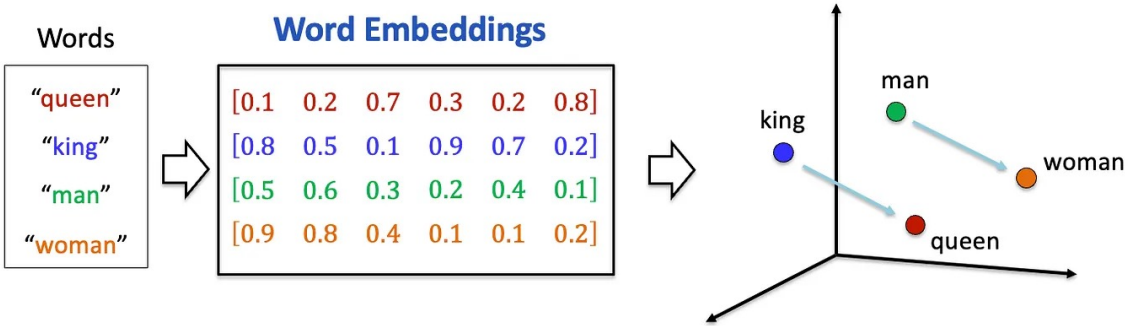
I	ate	an	apple
40	448	28397	30366

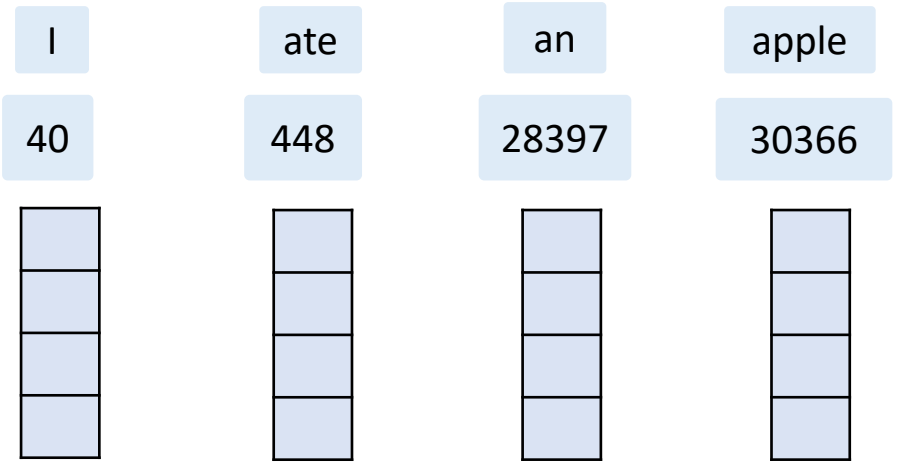
Embedding Layer

Generate Input Embeddings

Processing Inputs

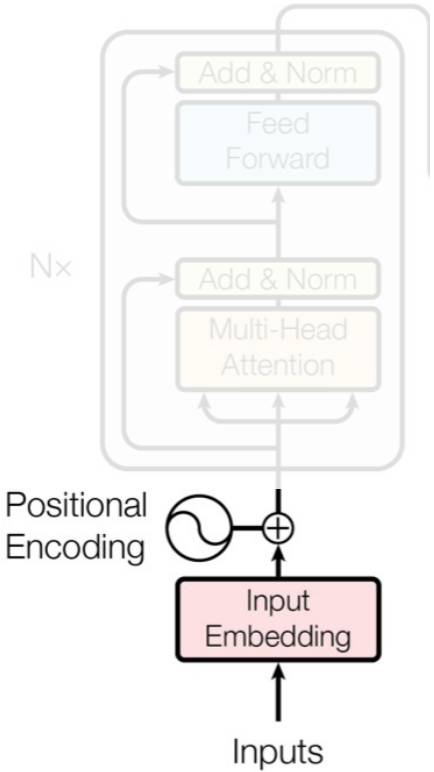


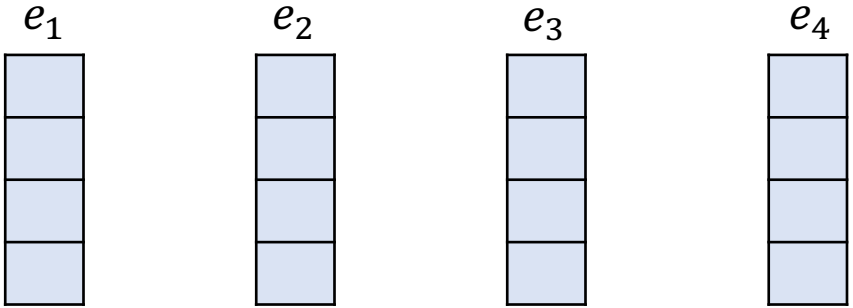




a vector of real numbers

Position Encodings

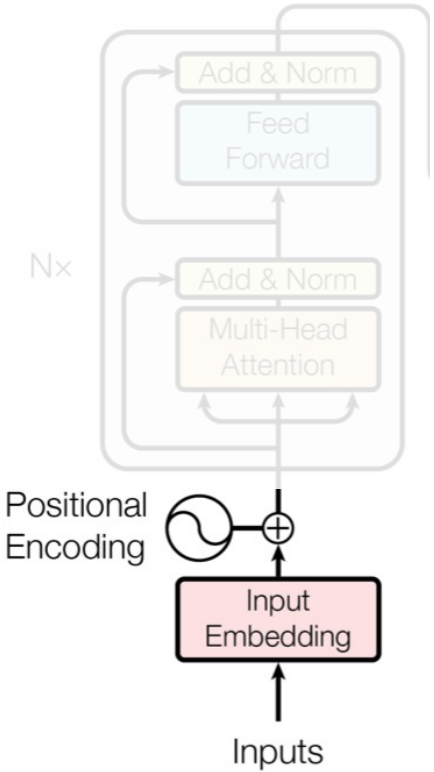


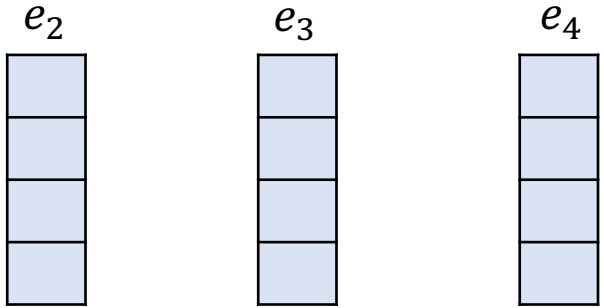




LSTM

Position Encodings

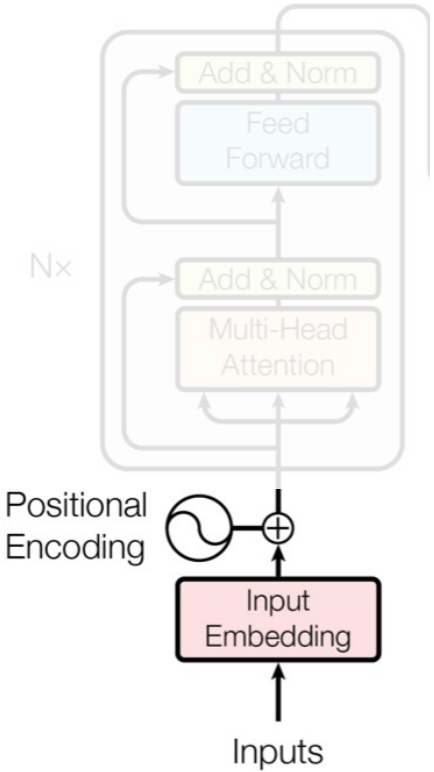


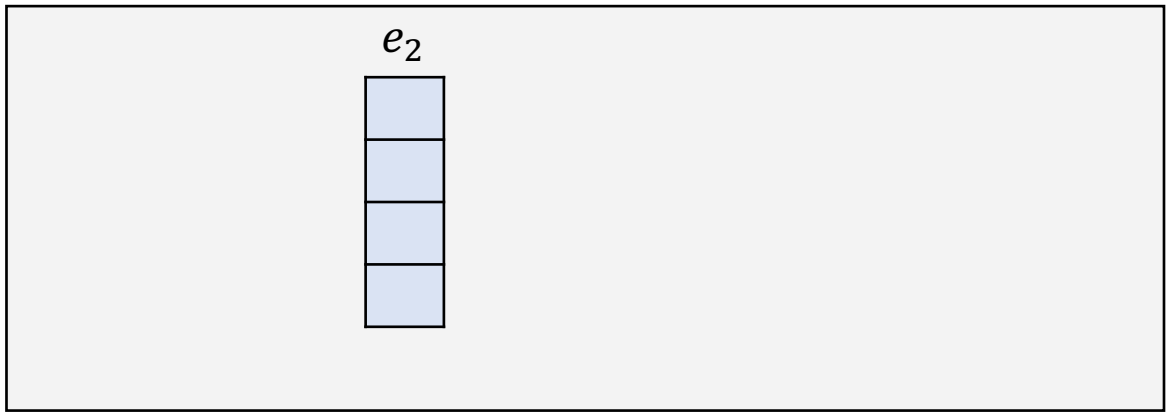




LSTM

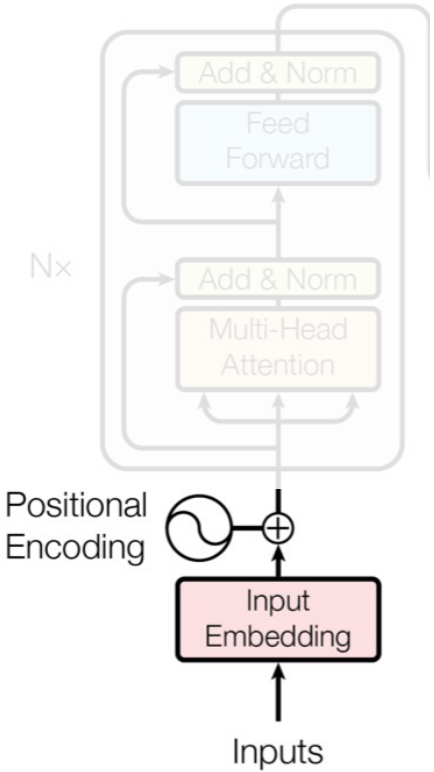
Position Encodings

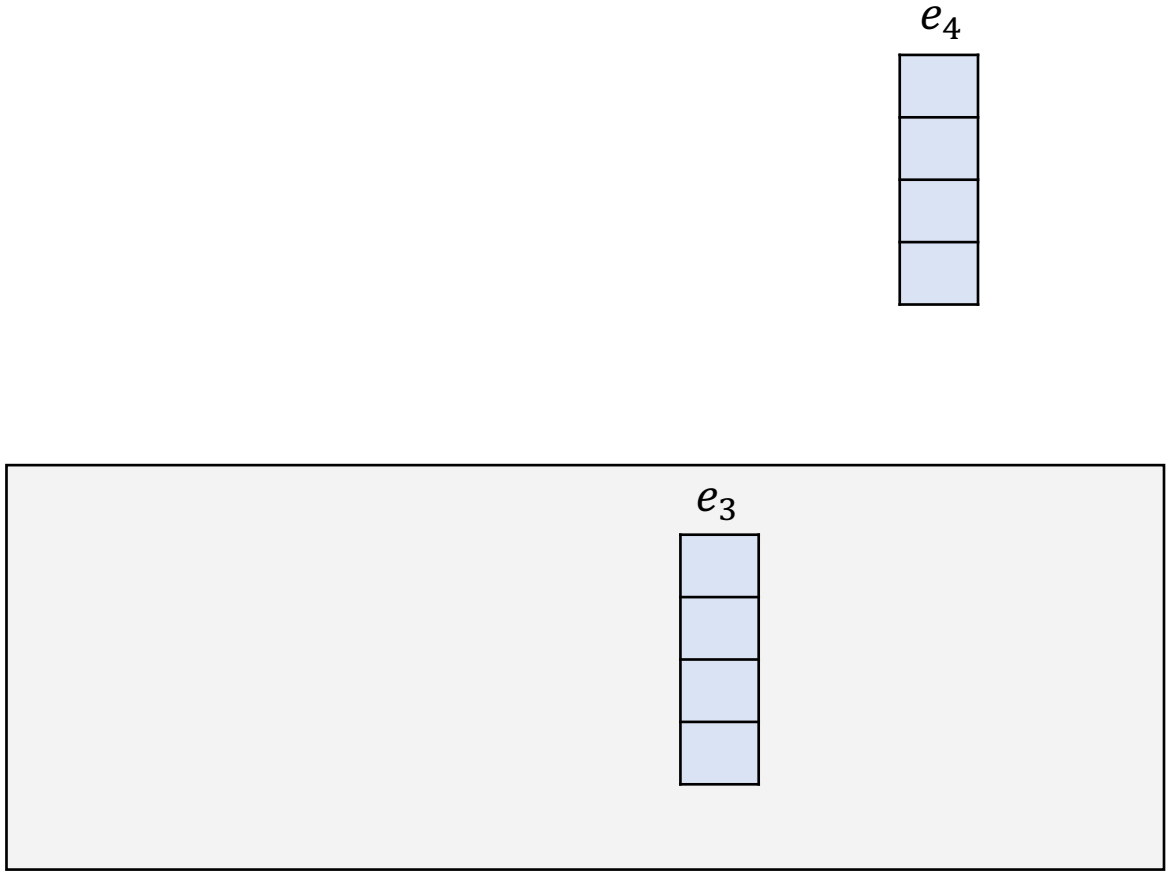




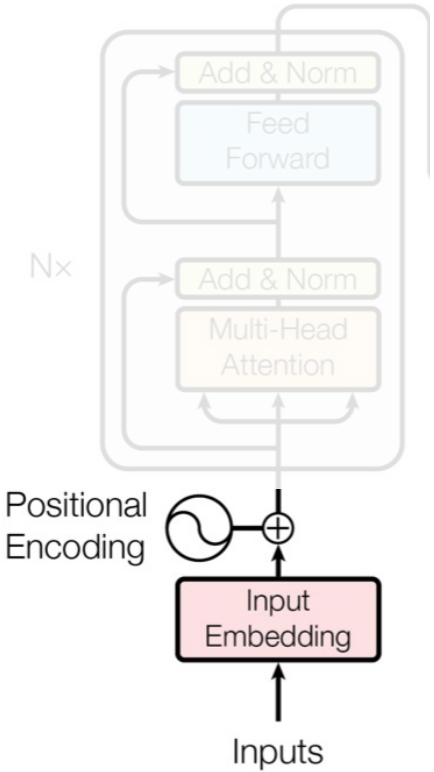
LSTM

Position Encodings





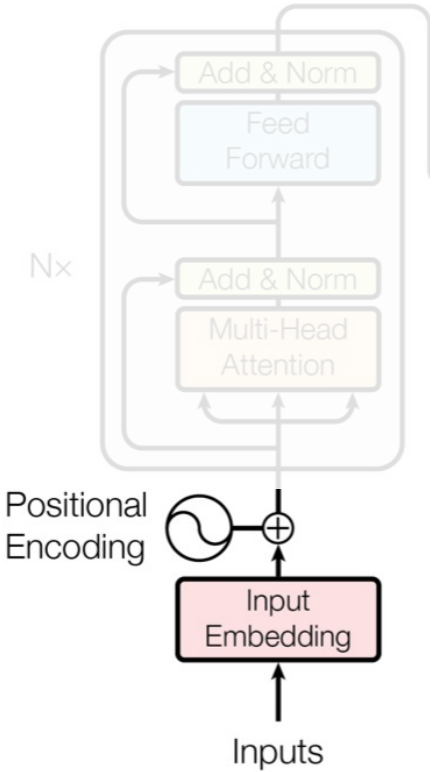
Position Encodings





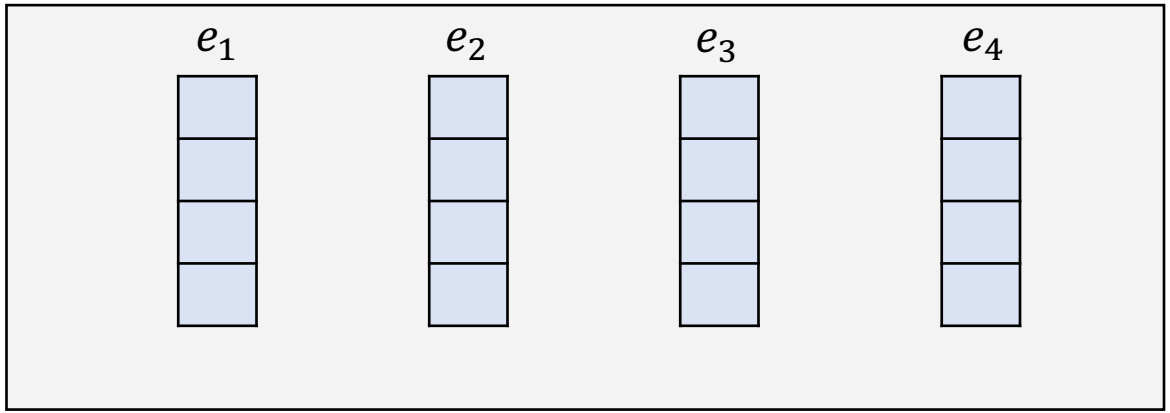
LSTM

Position Encodings



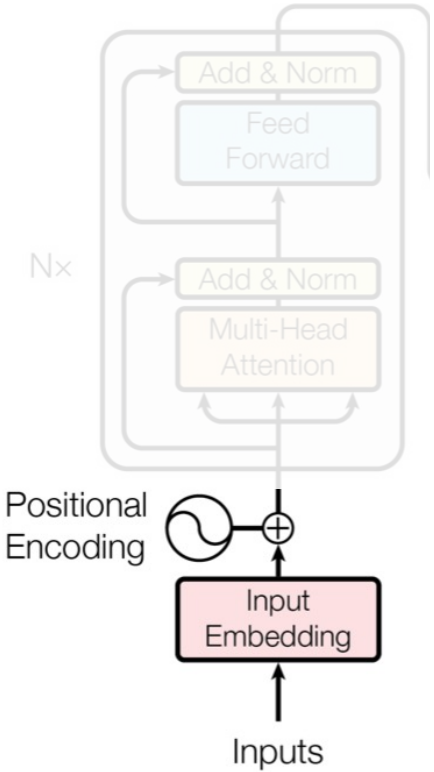
1 2 3 4

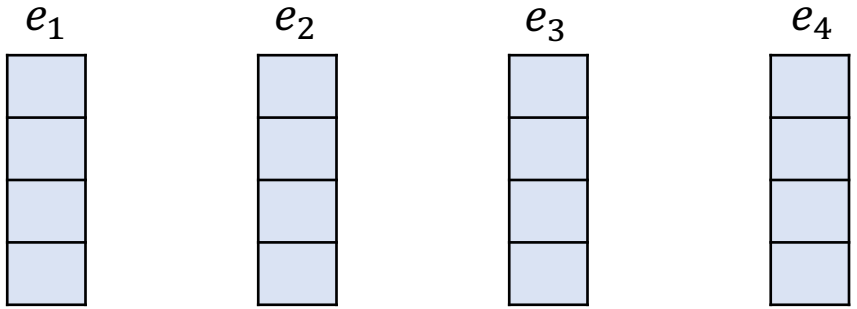
I ate an apple

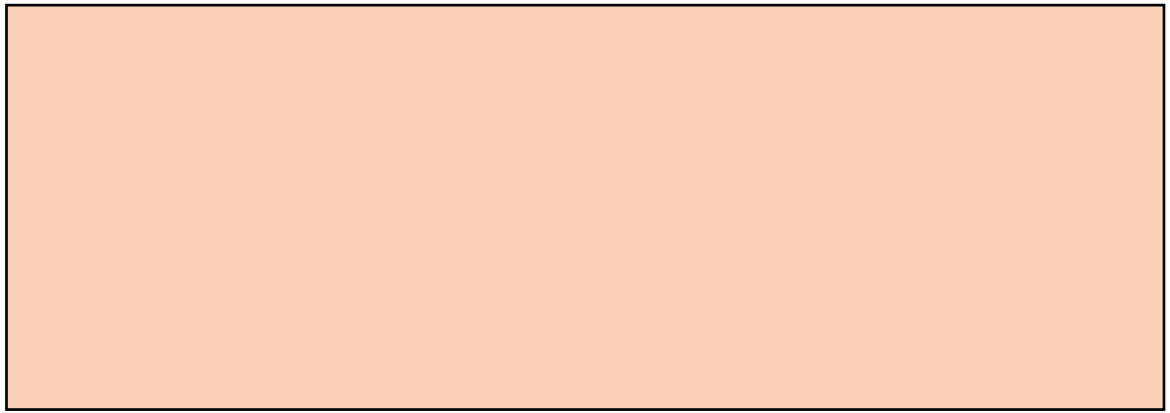


LSTM

Position Encodings

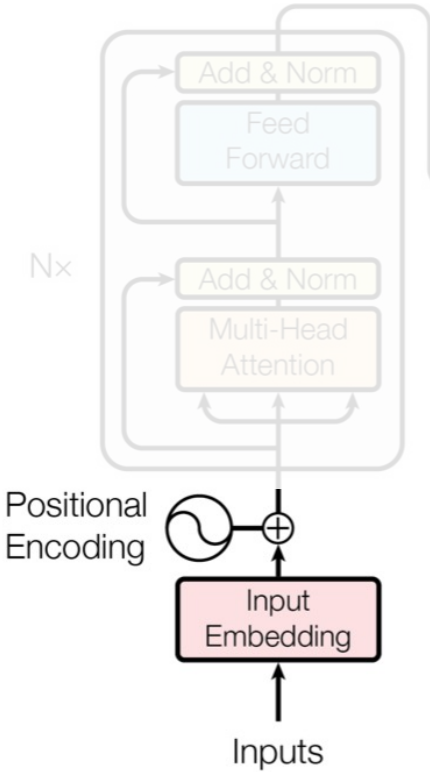


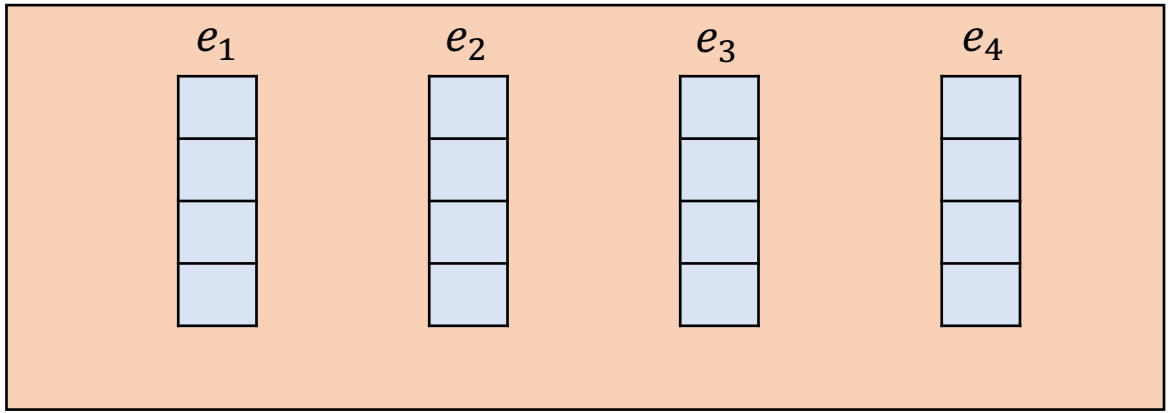




Transformer

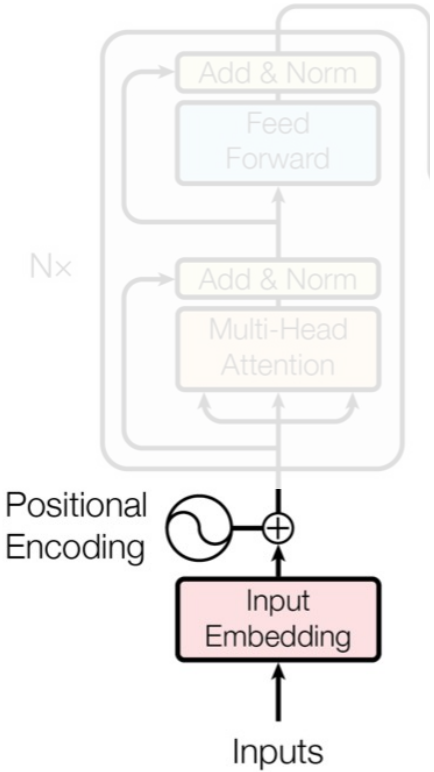
Position Encodings





Transformer

Position Encodings

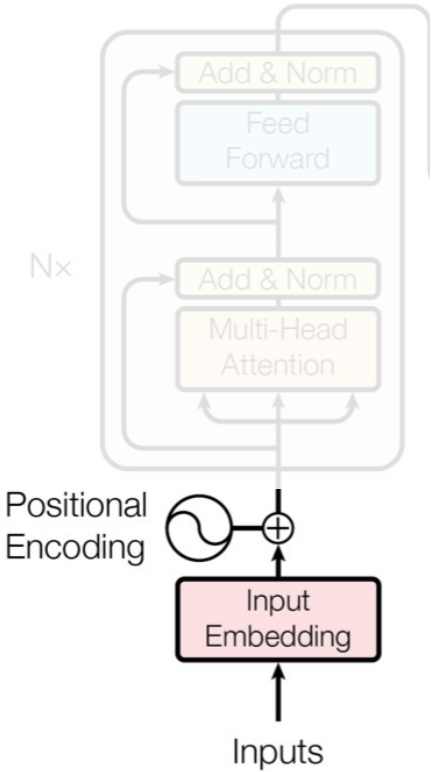


? ? ? ?

I ate an apple

Transformer

Position Encodings



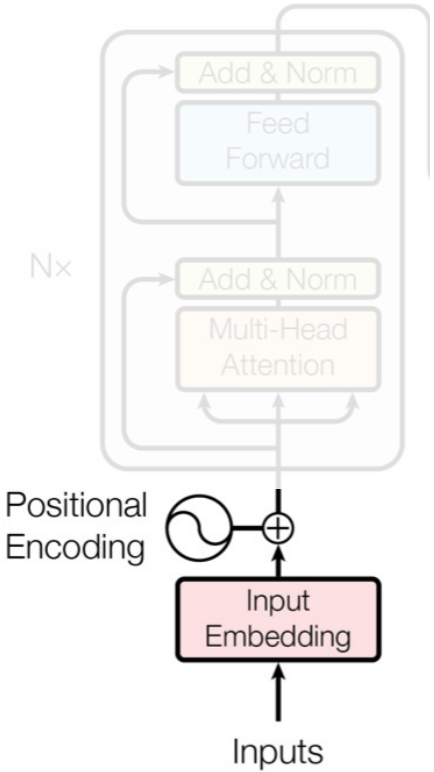
Why order matters?

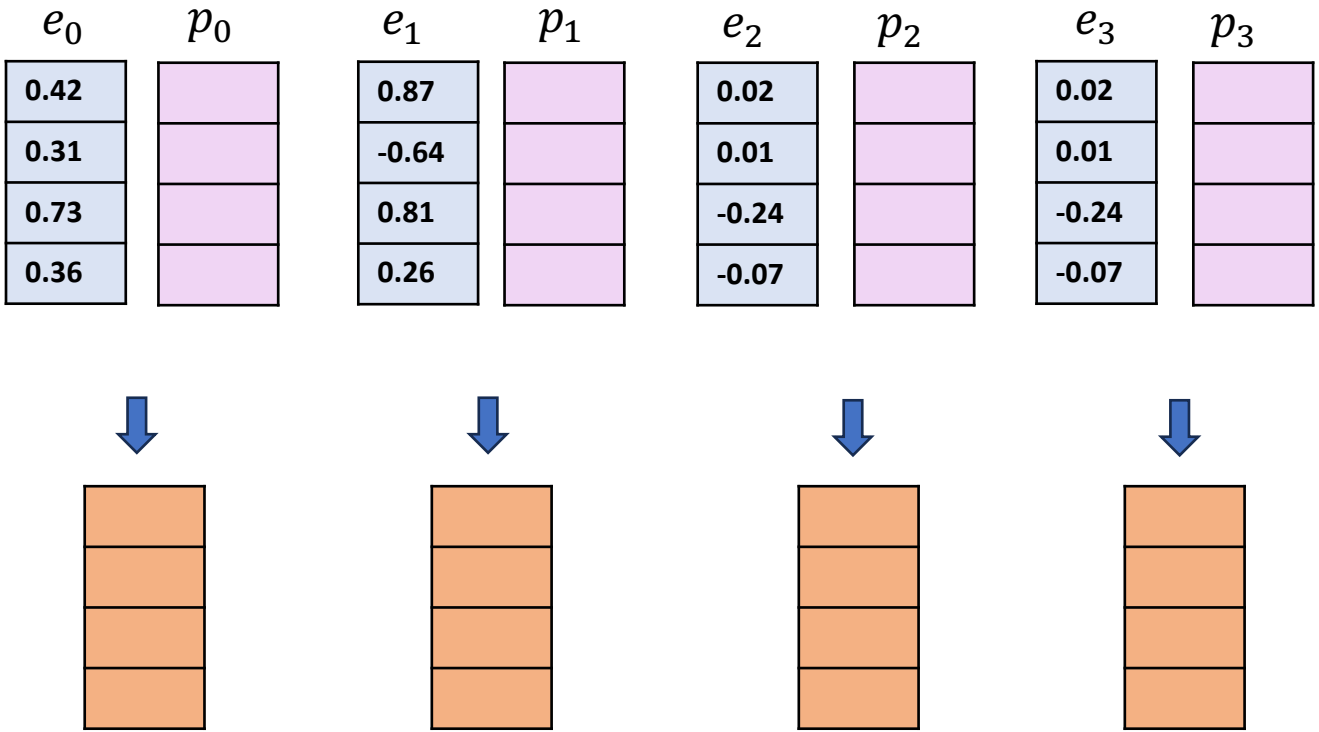
I ate an apple ✗ apple ate an I

Even though she did not win the award, she was satisfied. ✗

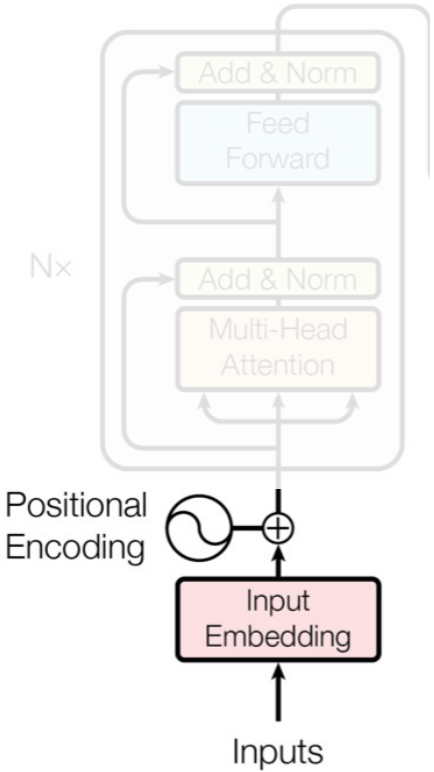
Even though she did win the award, she was not satisfied.

Position Encodings





Position Encodings



?

e_0	p_0
0.42	0
0.31	0
0.73	0
0.36	0

e_1	p_1
0.87	1
-0.64	1
0.81	1
0.26	1

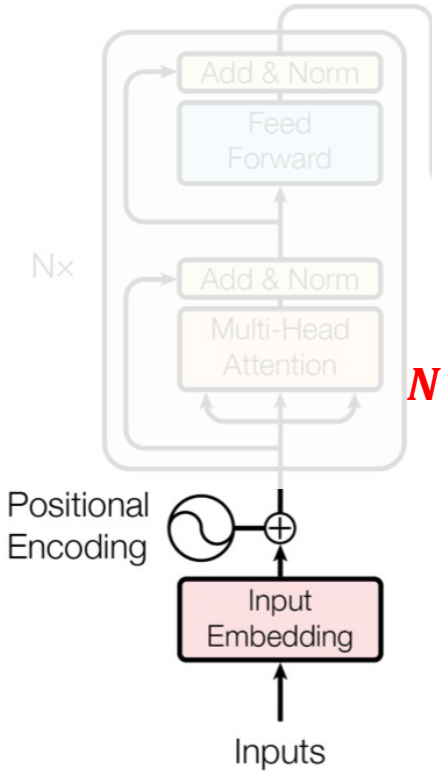
e_2	p_2
0.02	2
0.01	2
-0.24	2
-0.07	2

e_3	p_3
0.02	3
0.01	3
-0.24	3
-0.07	3

...

e_{30}	p_{30}
0.02	30
0.01	30
-0.24	30
-0.07	30

Position Encodings



$N = 4$

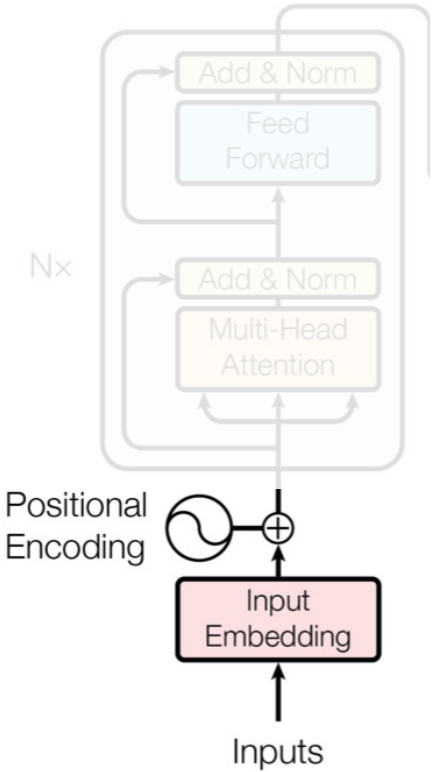
e_0	p_0	e_1	p_1	e_2	p_2	e_3	p_3
0.42	0	0.87	0.33	0.02	0.66	0.02	1
0.31	0	-0.64	0.33	0.01	0.66	0.01	1
0.73	0	0.81	0.33	-0.24	0.66	-0.24	1
0.36	0	0.26	0.33	-0.07	0.66	-0.07	1

$N = 5$

0.42	0	0.87	0.2	0.02	0.4		0.02	1
0.31	0	-0.64	0.2	0.01	0.4		0.01	1
0.73	0	0.81	0.2	-0.24	0.4	...	-0.24	1
0.36	0	0.26	0.2	-0.07	0.4		-0.07	1

e_4 p_4

Position Encodings



e_0	p_0	e_1	p_1	e_2	p_2	e_3	p_3
0.42	?	0.87	?	0.02	?	0.02	?
0.31	?	-0.64	?	0.01	?	0.01	?
0.73	?	0.81	?	-0.24	?	-0.24	?
0.36	?	0.26	?	-0.07	?	-0.07	?

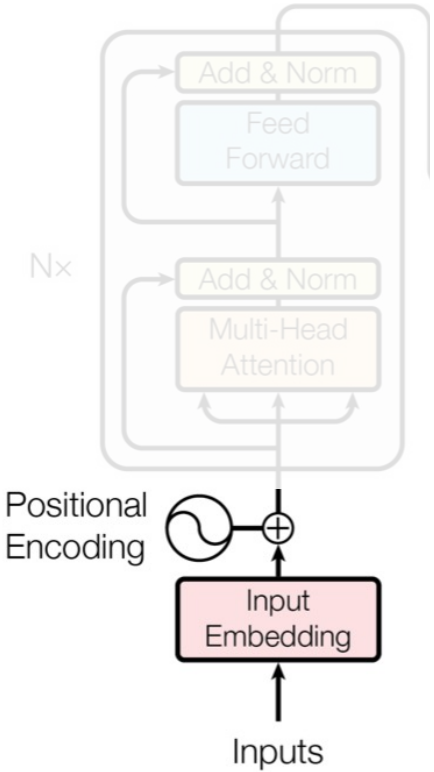
$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

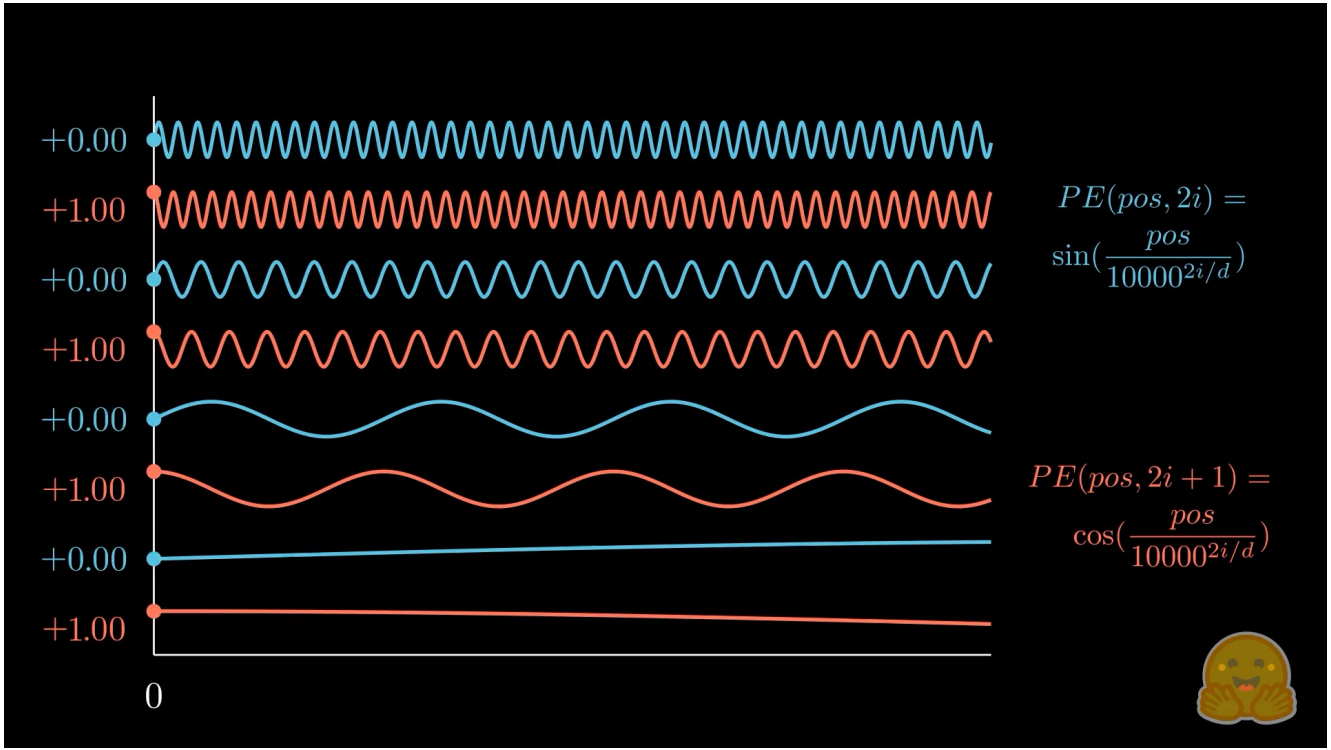
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

	p_0
$i = 0$	?
$i = 1$	?
$i = 2$	?
$i = 3$	?

$d = 4$

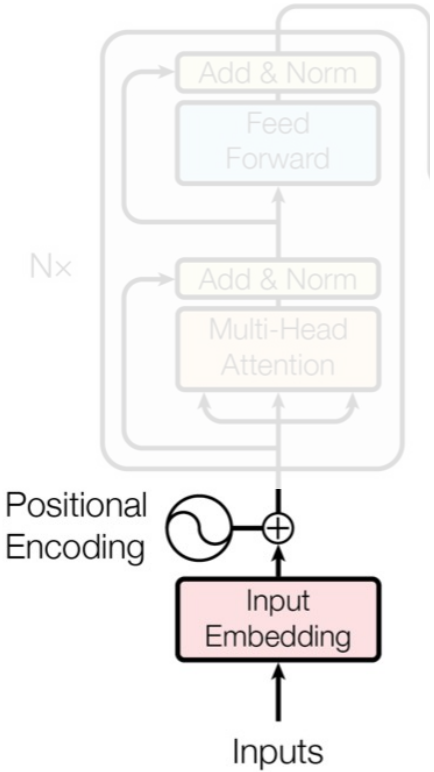
Position Encodings





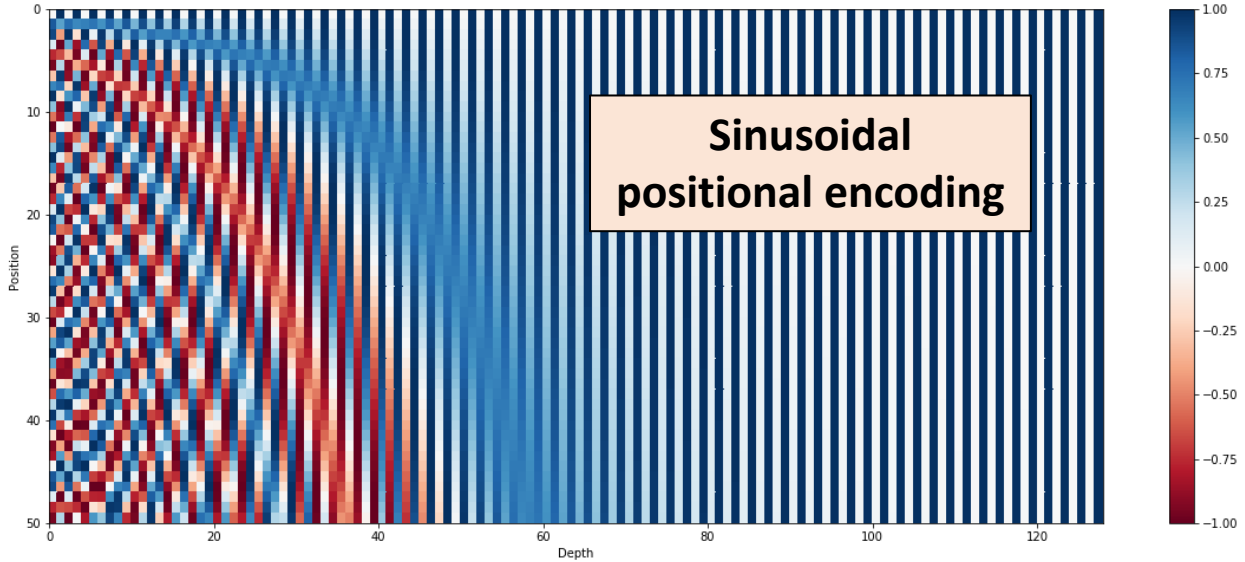
https://huggingface.co/blog/designing-positional-encoding

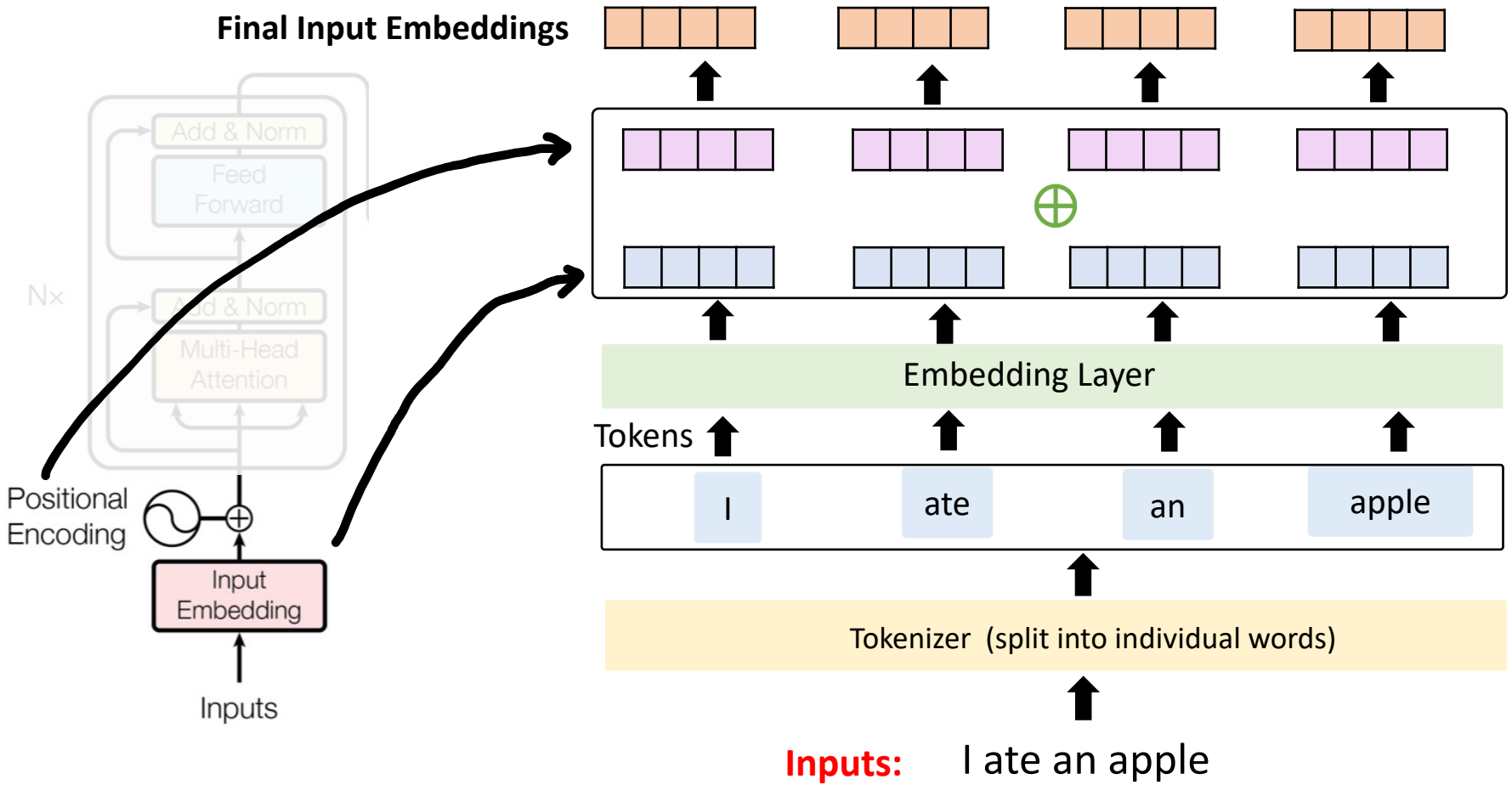
Position Encodings



Binary positional encoding

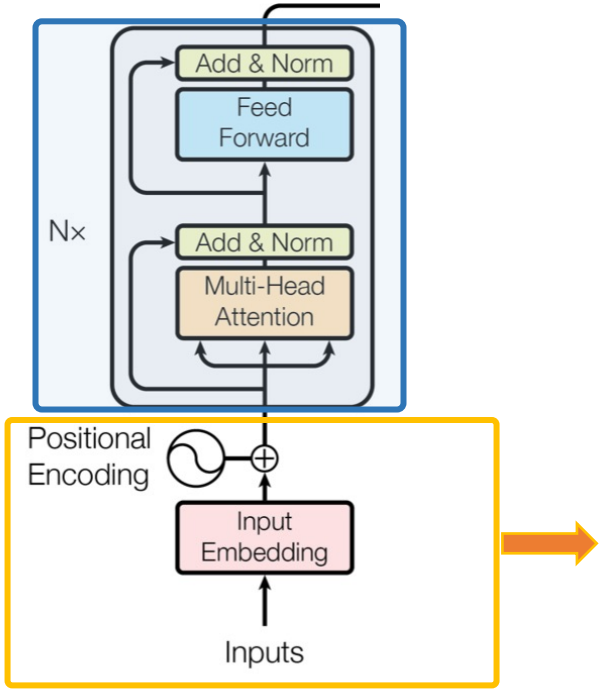
0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1




Final Input Embeddings
Add & Norm
Feed Forward
Nx
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Inputs
Embedding Layer
Tokens
I
ate
an
apple
Tokenizer (split into individual words)
Inputs: I ate an apple

Encoder

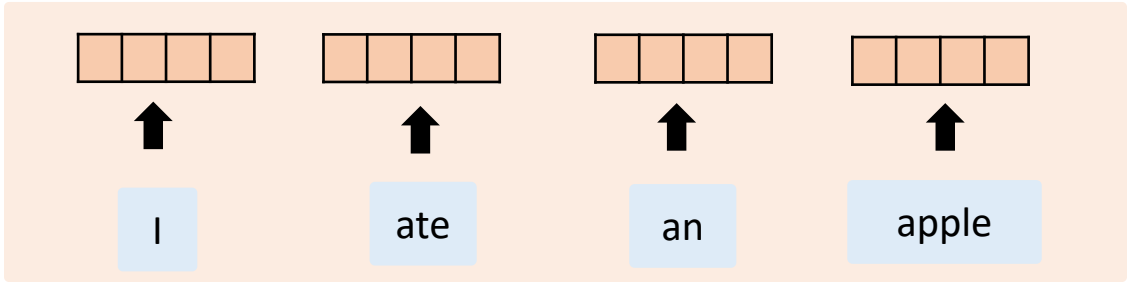
The encoder turns a sequence of tokens into a sequence of contextual representations, where each token vector “knows” about all other tokens in the sequence



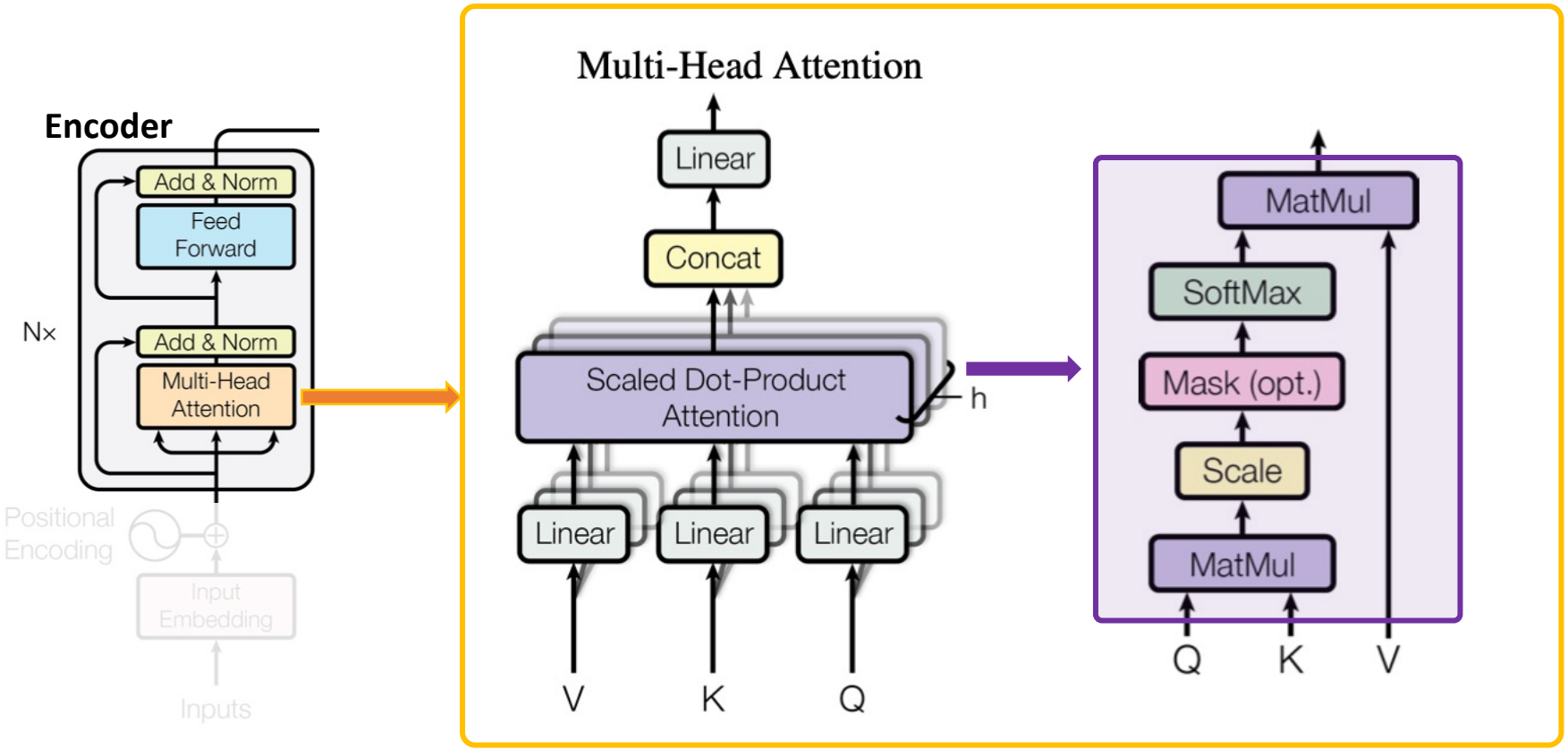
CONTEXTUALLY RICH EMBEDDINGS



WHERE IS THE CONTEXT ?

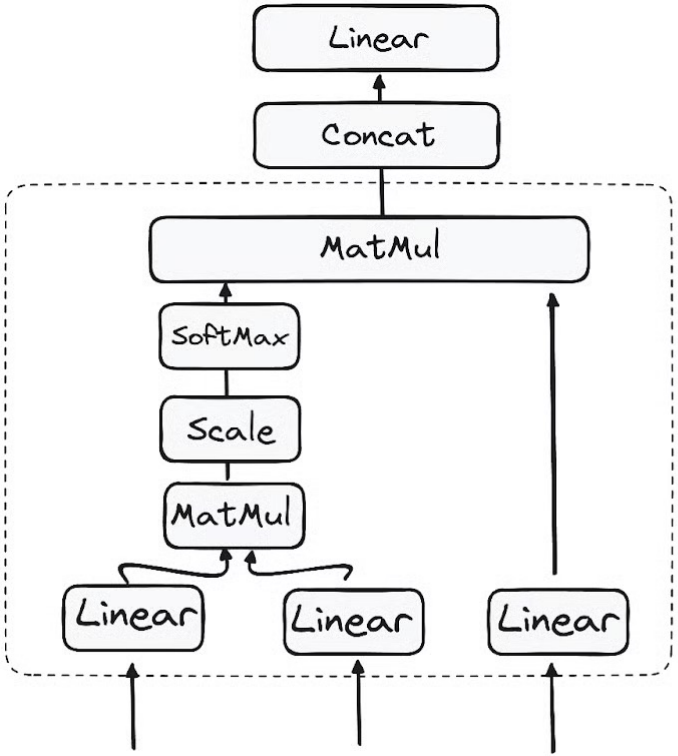


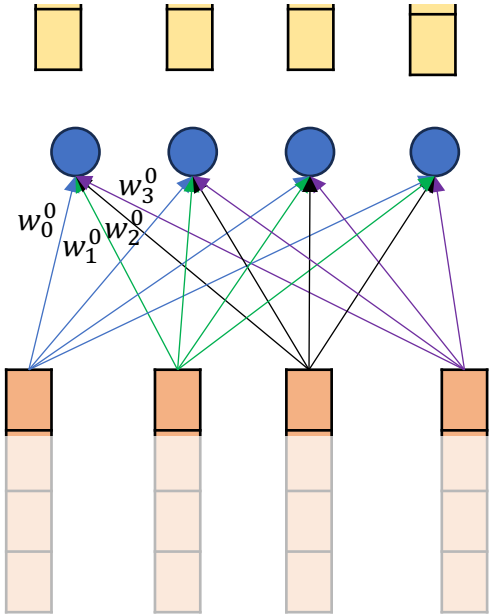
Multi-Head Attention



Multi-Head Attention

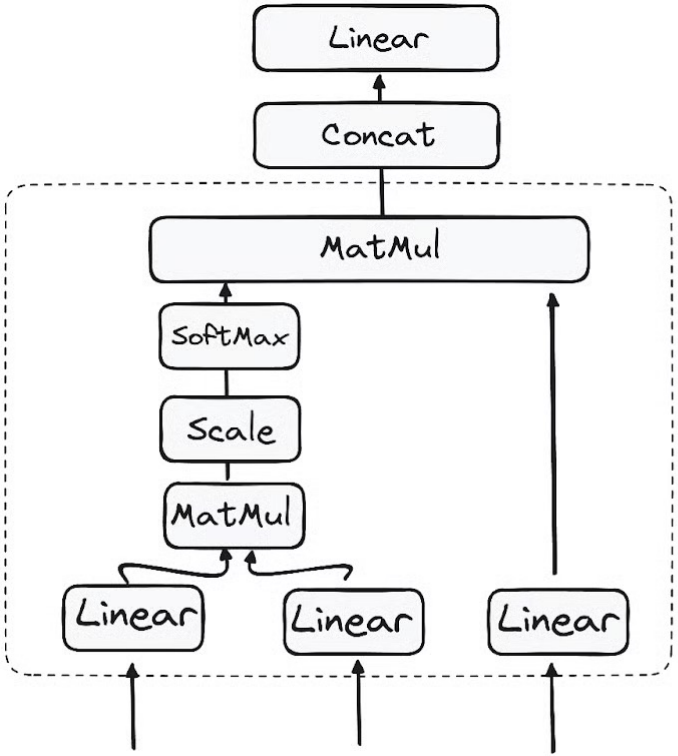
Self-Attention

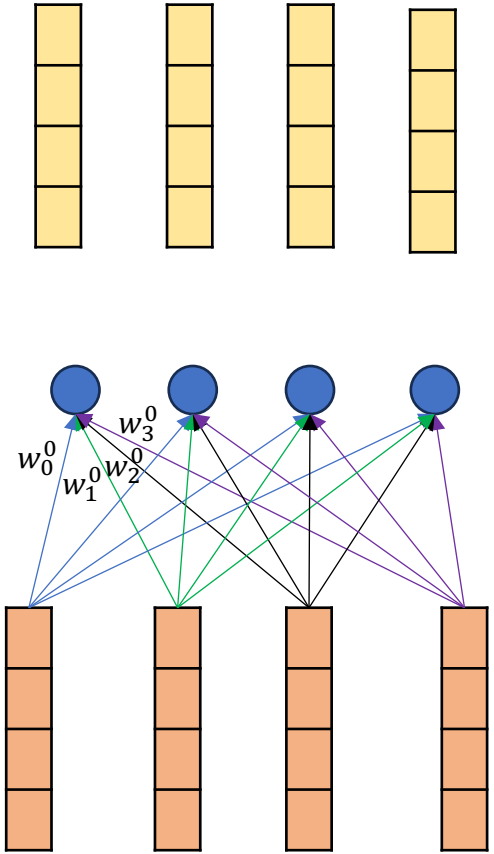




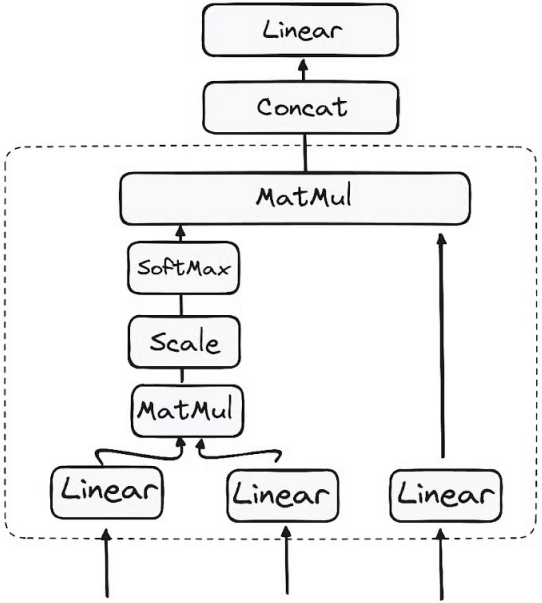
Multi-Head Attention

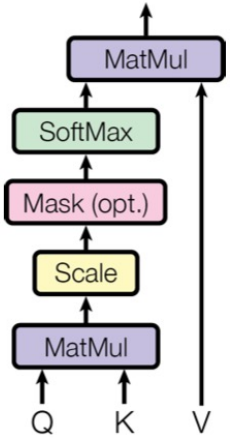
Self-Attention

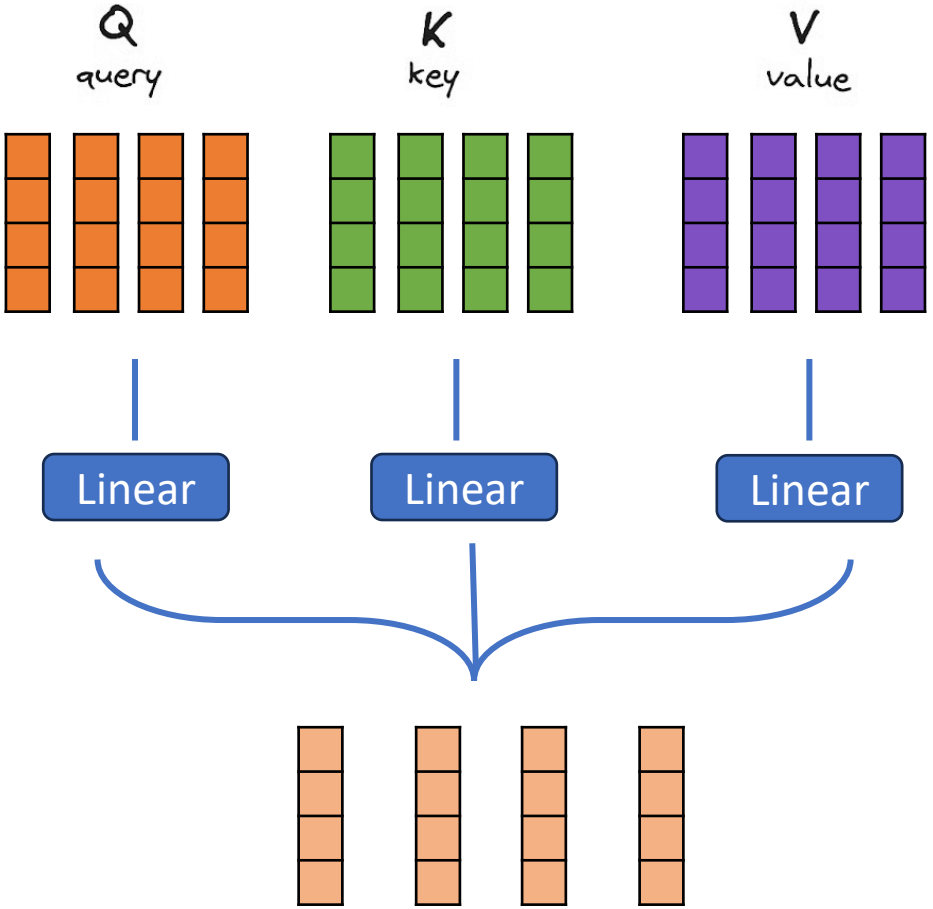




Multi-Head Attention

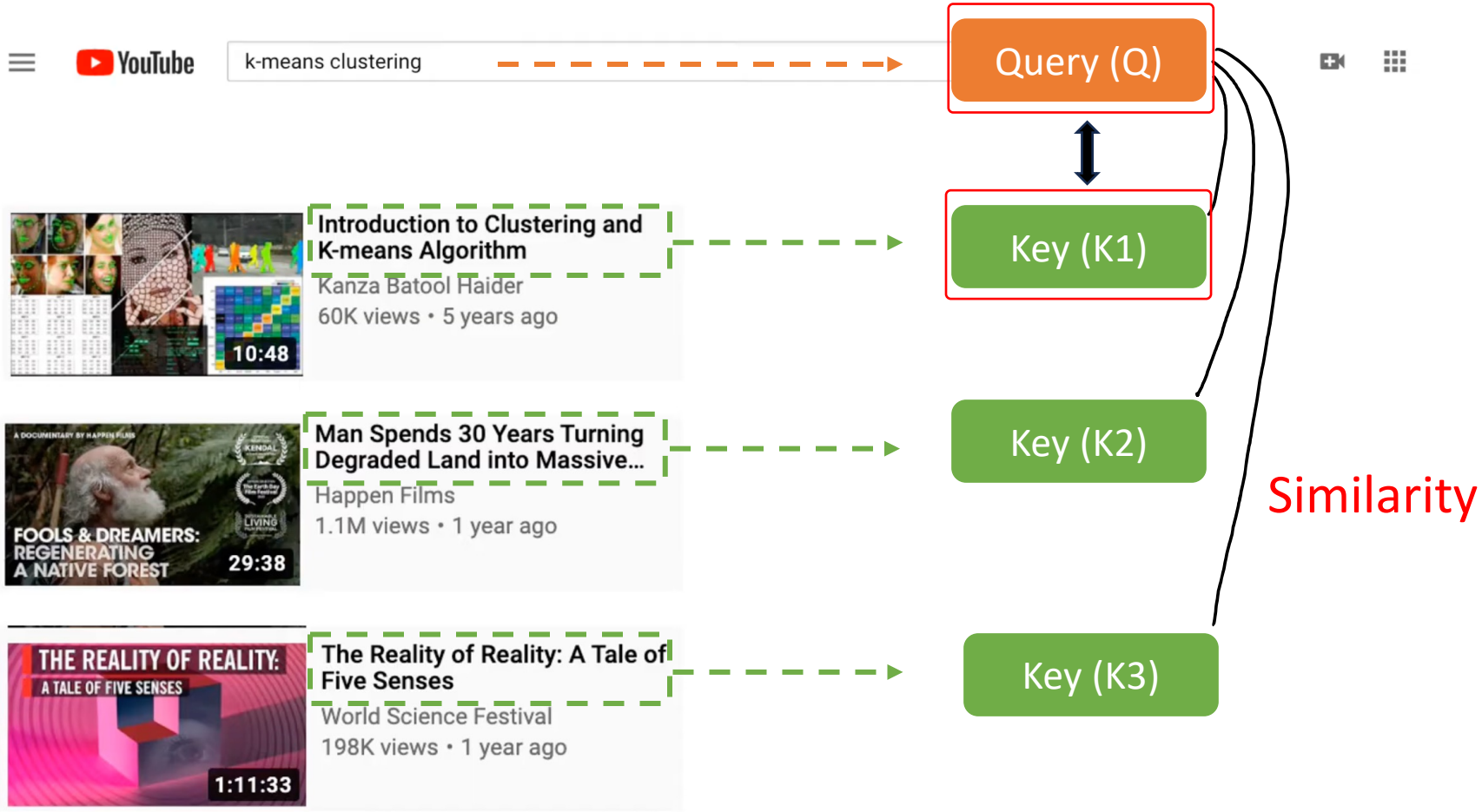






I ate an apple

Query, Key and Value



Query, Key and Value



Query

1. Search for info

Key

1. Interacts directly with Queries
2. Distinguishes one object from another
3. Identify which object is the most relevant and by how much

Value

1. Actual details of the object
2. More fine grained

Similarity

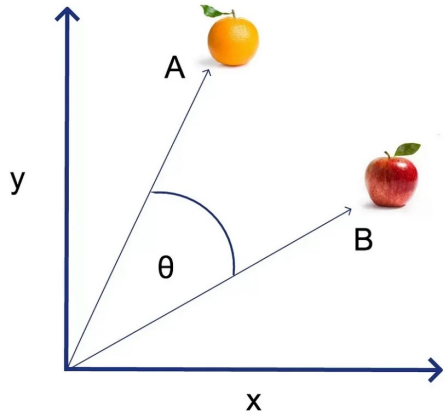
Query (Q)

Similarity?

Key (K1)

similarity $(Q, K) = \frac{Q \,.\, K^T}{scaling}$

Cosine Similarity



$$\text{Cos}\,(A, B) = \frac{A \,.\, B}{|A|\,|B|}$$

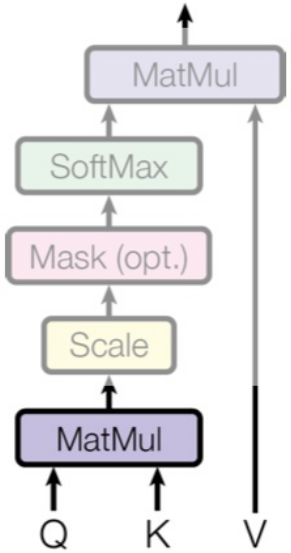
Vectors

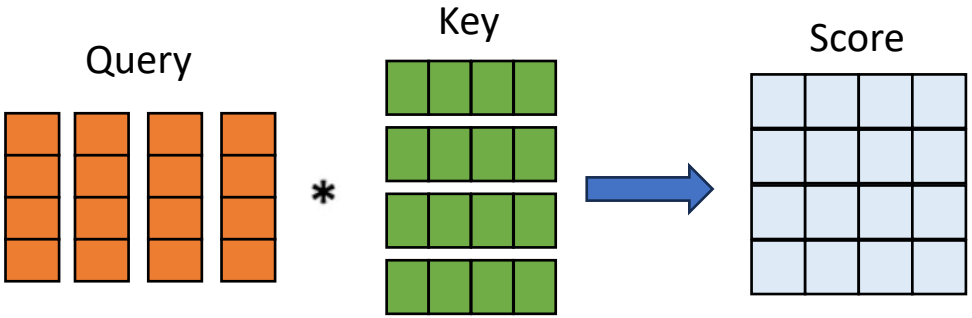
$$\text{similarity}\,(A, B) = \frac{A \,.\, B}{scaling}$$

Matrices

$$\text{similarity}\,(A, B) = \frac{A \,.\, B^T}{scaling}$$

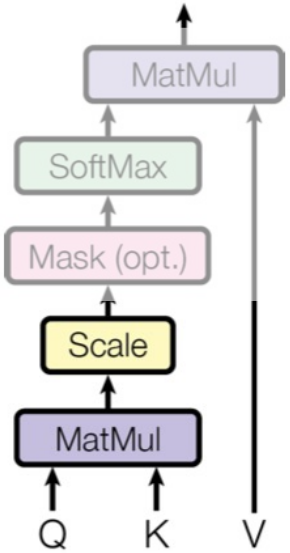
Self-Attention

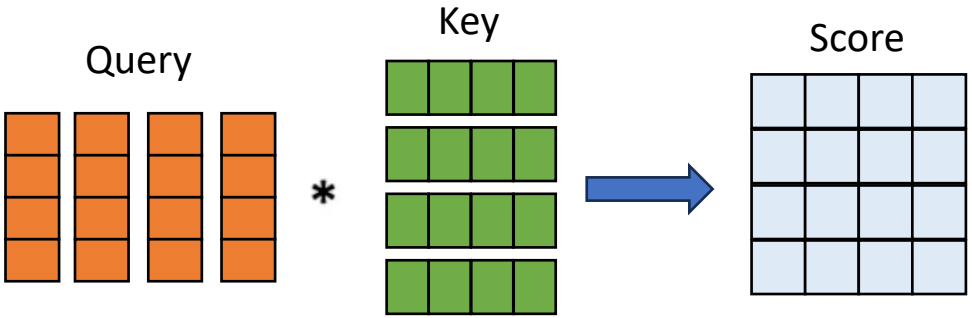


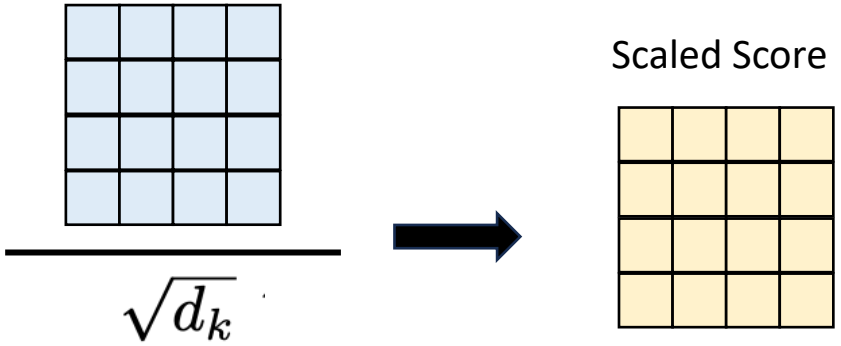


	I	ate	an	apple
I	**98**	**27**	**10**	**12**
ate	**27**	**89**	**31**	**67**
an	**10**	**31**	**91**	**54**
apple	**12**	**67**	**54**	**92**

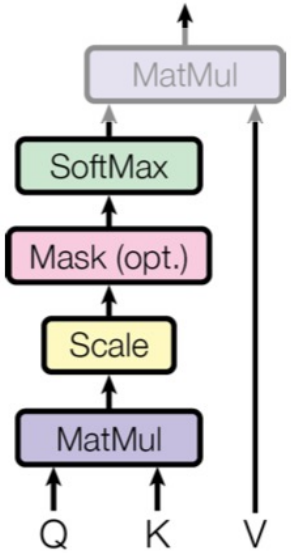
Self-Attention







Self-Attention



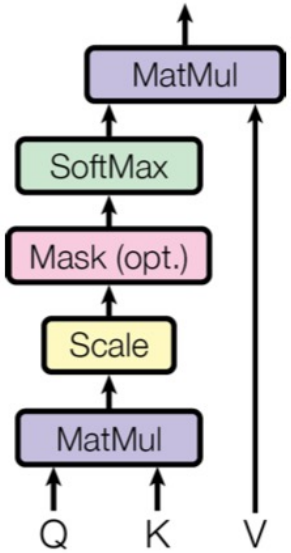
Scaled Score

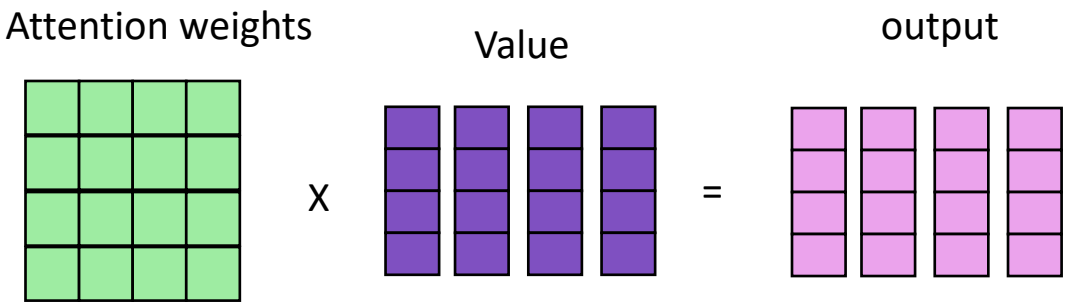
SoftMax

=

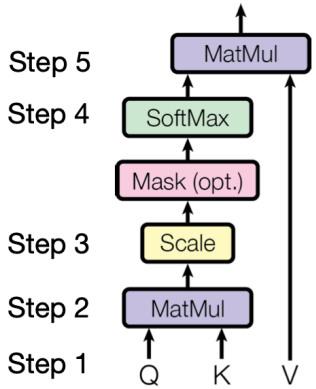
	I	ate	an	apple
I	0.7	0.1	0.1	0.1
ate	0.1	0.6	0.2	0.1
an	0.1	0.3	0.6	0.1
apple	0.1	0.3	0.3	0.3

Self-Attention



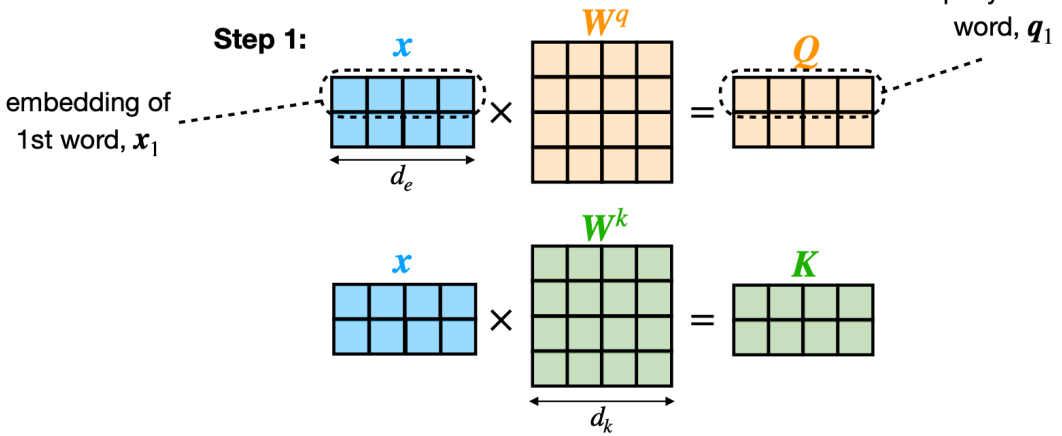


Scaled Dot-Product Attention

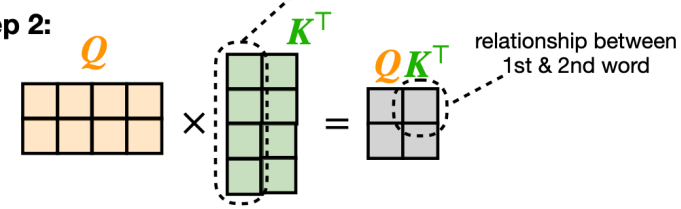


Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., 2017. Attention Is All You Need.

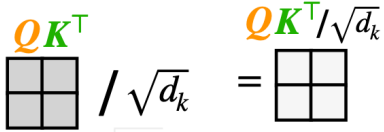
Step 1:



Step 2:

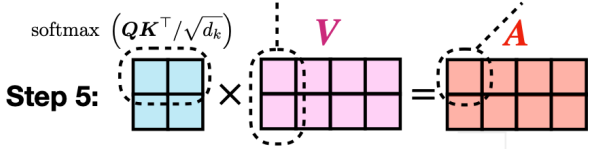


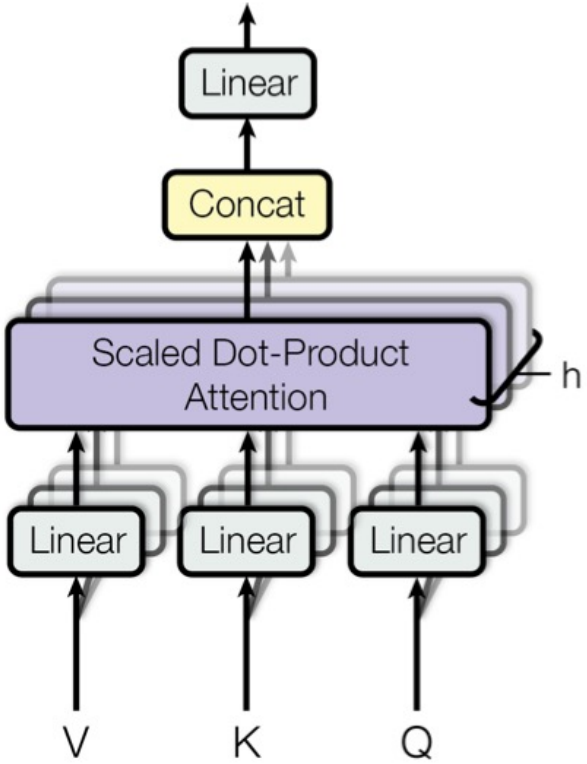
Step 3:



Step 4: $\text{softmax}\left(QK^\top/\sqrt{d_k}\right) =$

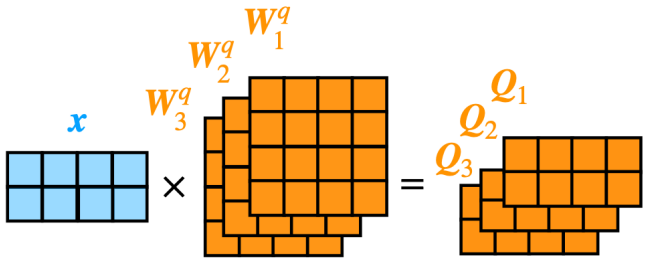
Step 5:

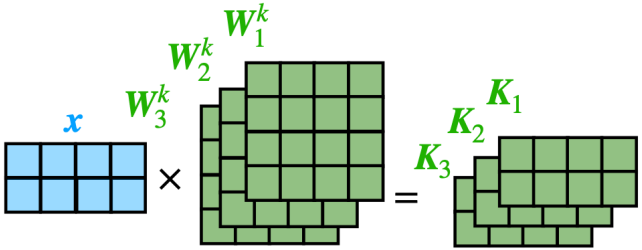


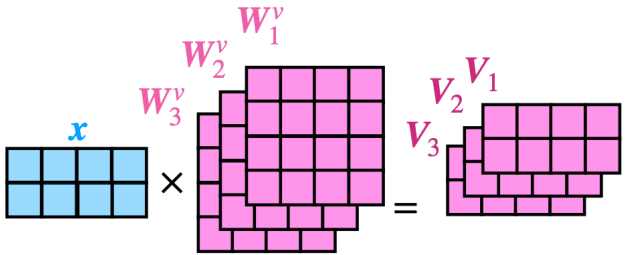


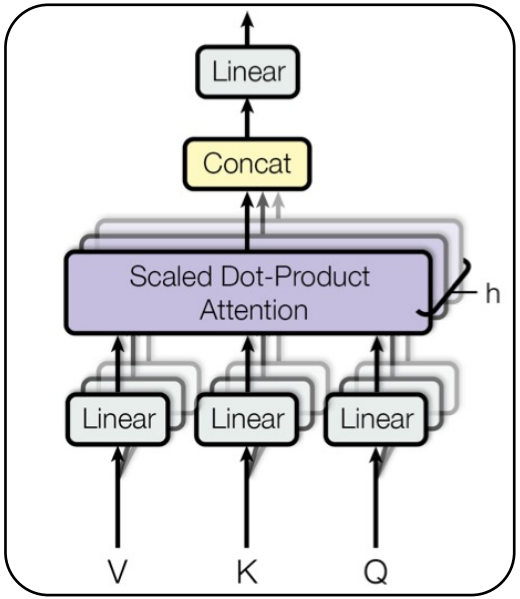
Self-attention head

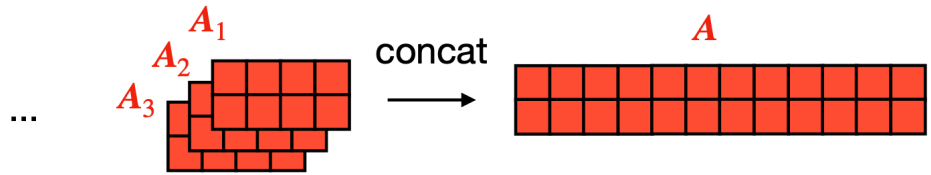
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

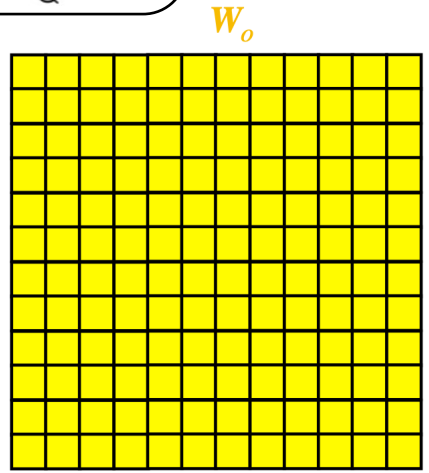

W_1^q
W_2^q
W_3^q
x
Q_1
Q_2
Q_3


W_1^k
W_2^k
W_3^k
x
K_1
K_2
K_3

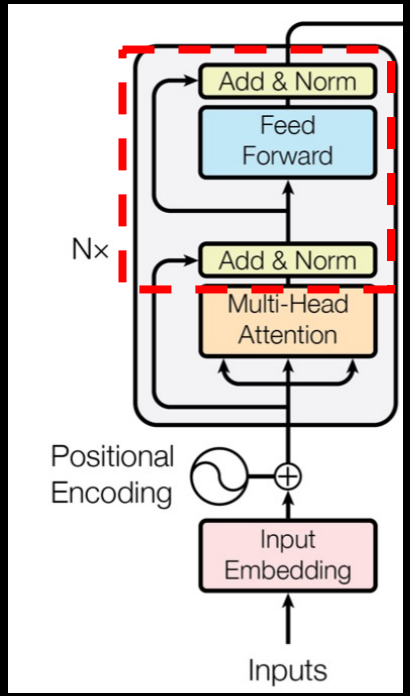

W_1^v
W_2^v
W_3^v
x
V_1
V_2
V_3

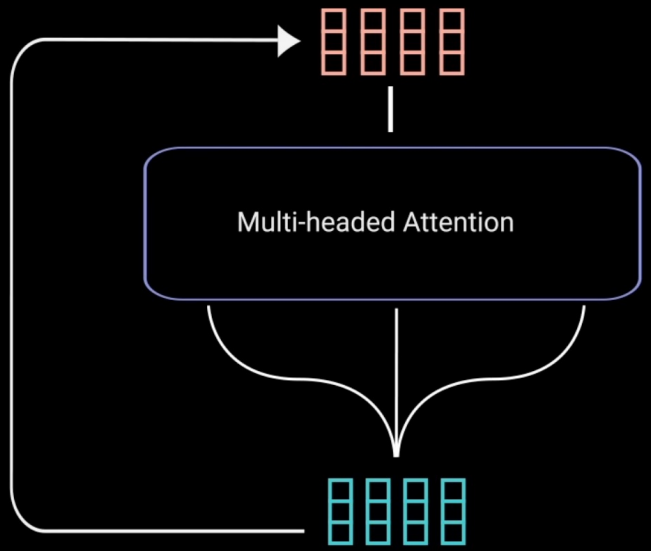

Linear
Concat
Scaled Dot-Product Attention
h
Linear
Linear
Linear
V
K
Q

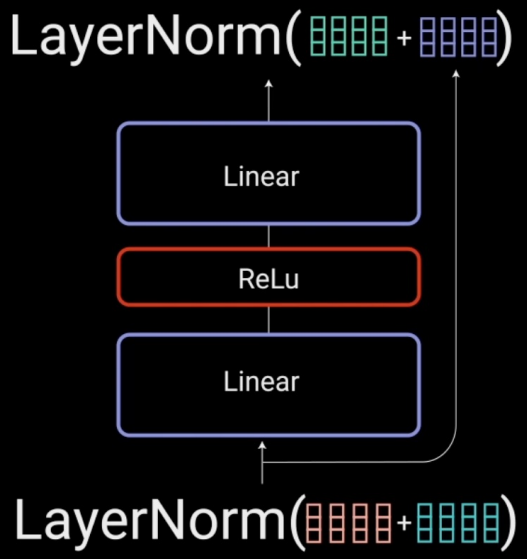

...
A_1
A_2
A_3
concat
A


W_o

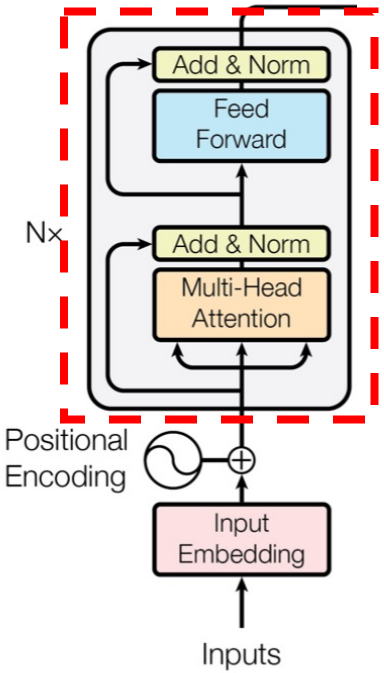
Residual Connection, Layer Normalization & Feed Forward



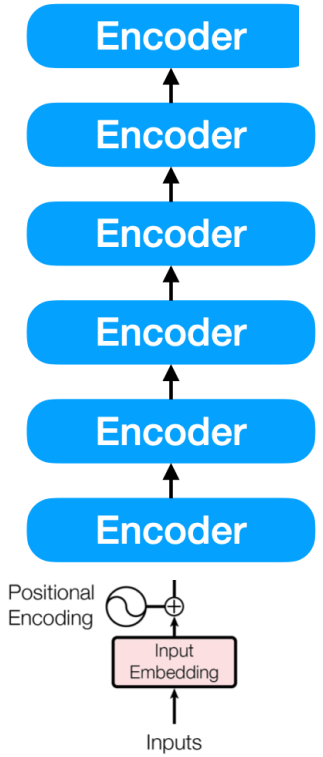


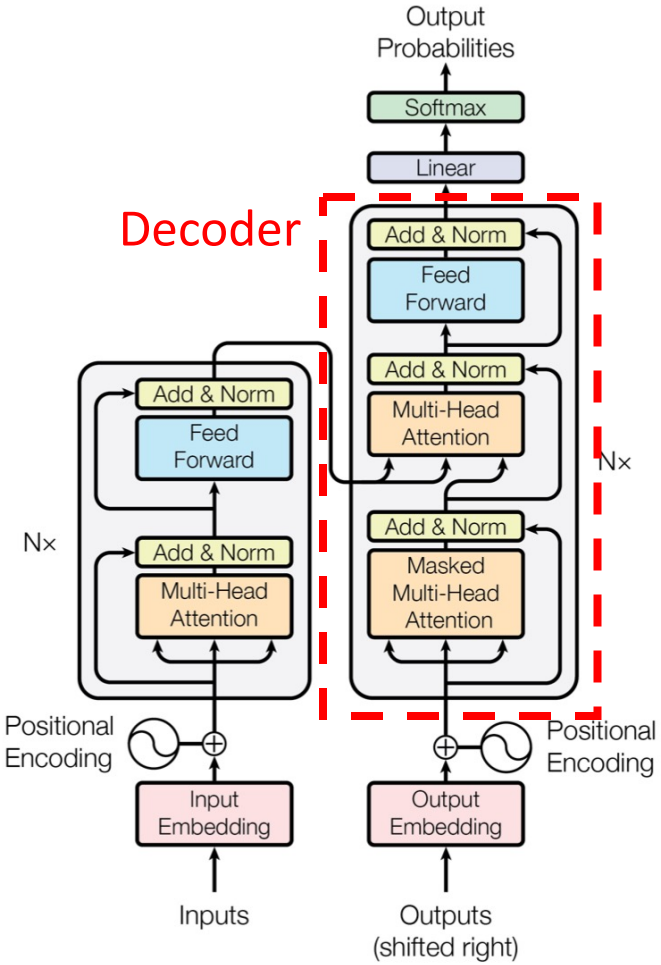


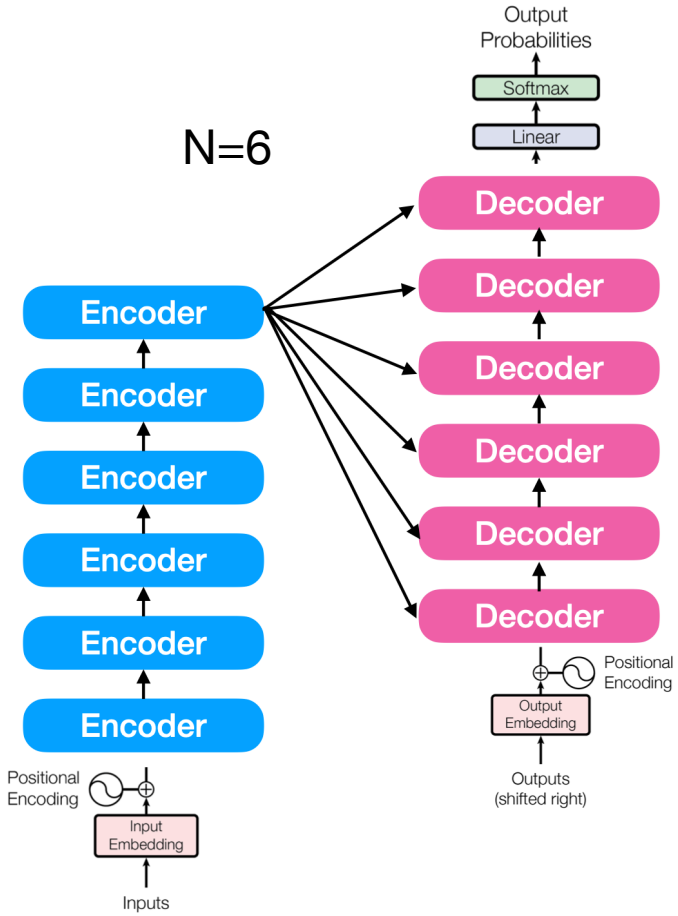
Encoder layer

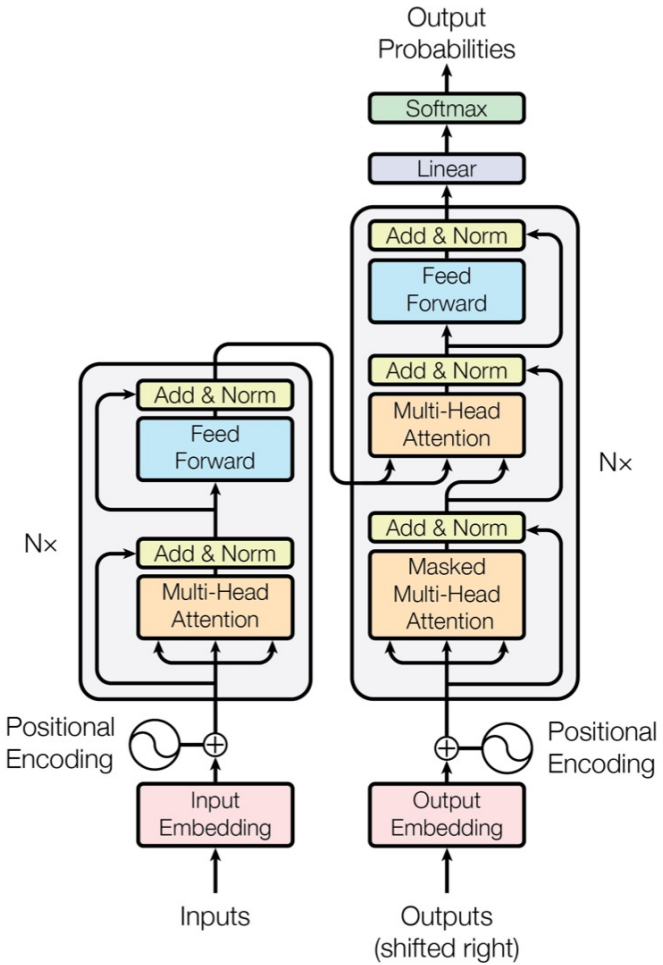


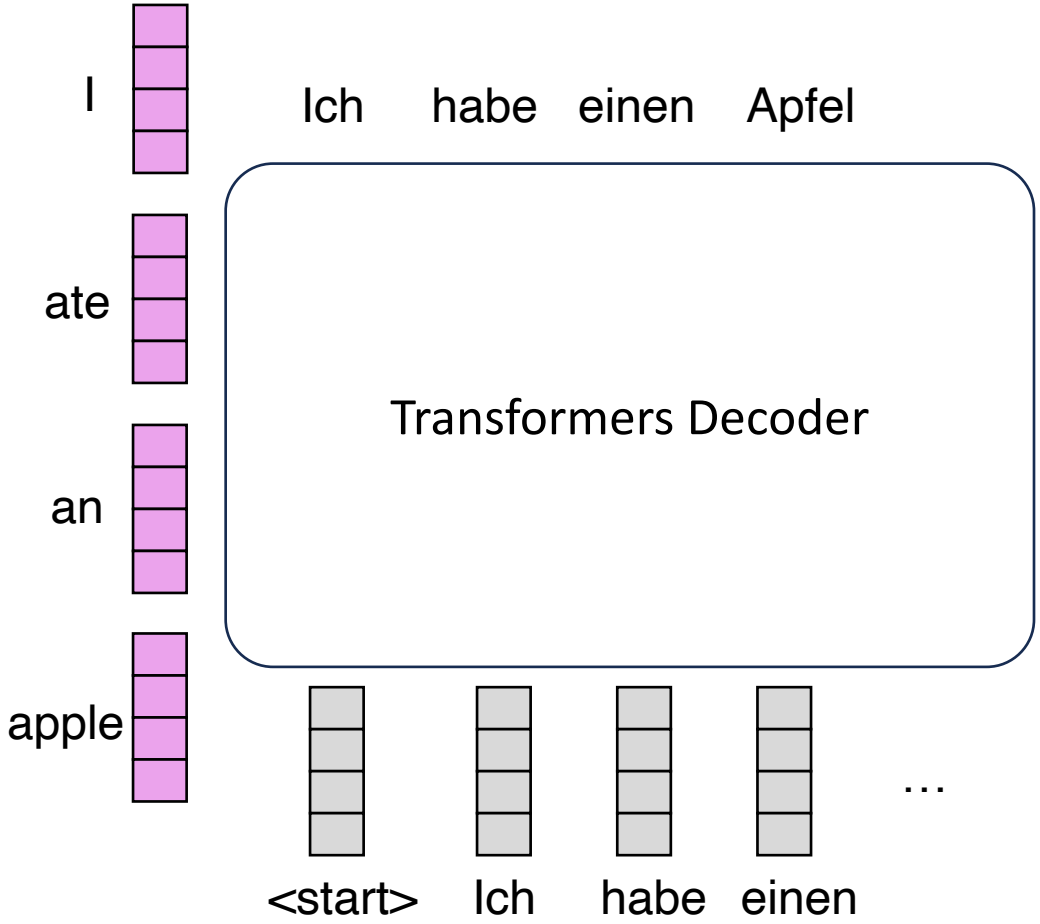
Encoder (6x)

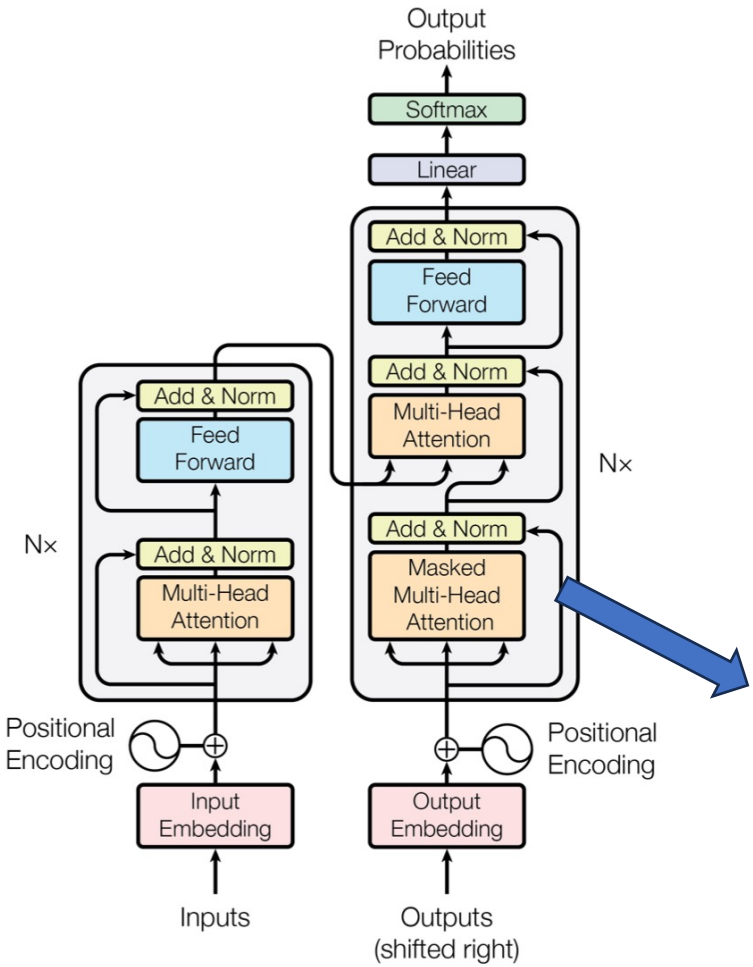



Output Probabilities
Softmax
Linear
Decoder
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)


N=6
Output Probabilities
Softmax
Linear
Encoder
Encoder
Encoder
Encoder
Encoder
Encoder
Decoder
Decoder
Decoder
Decoder
Decoder
Decoder
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

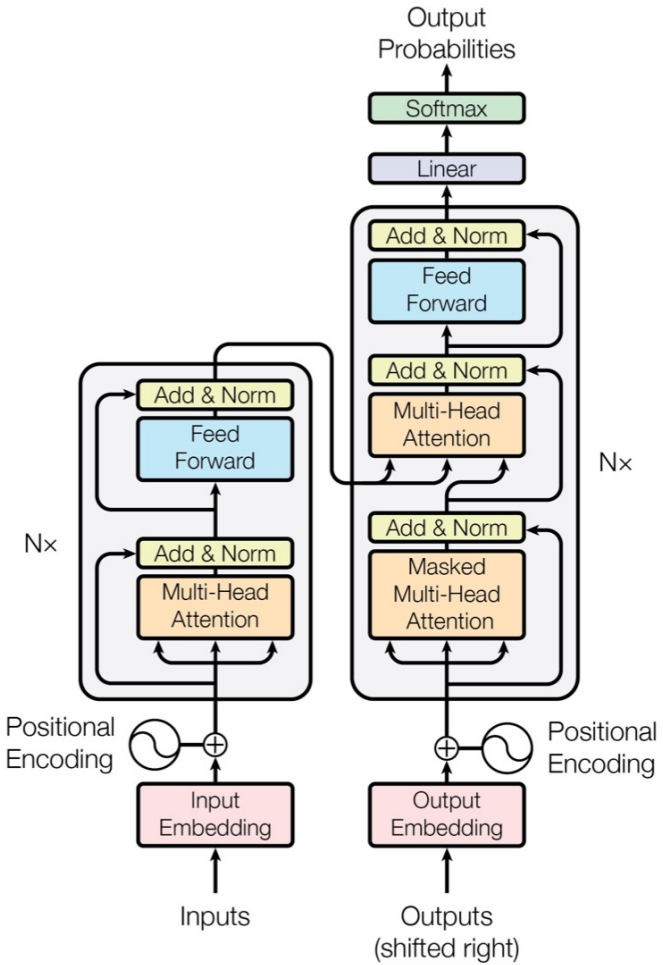

Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)


I
ate
an
apple
Ich habe einen Apfel
Transformers Decoder
<start> Ich habe einen
...



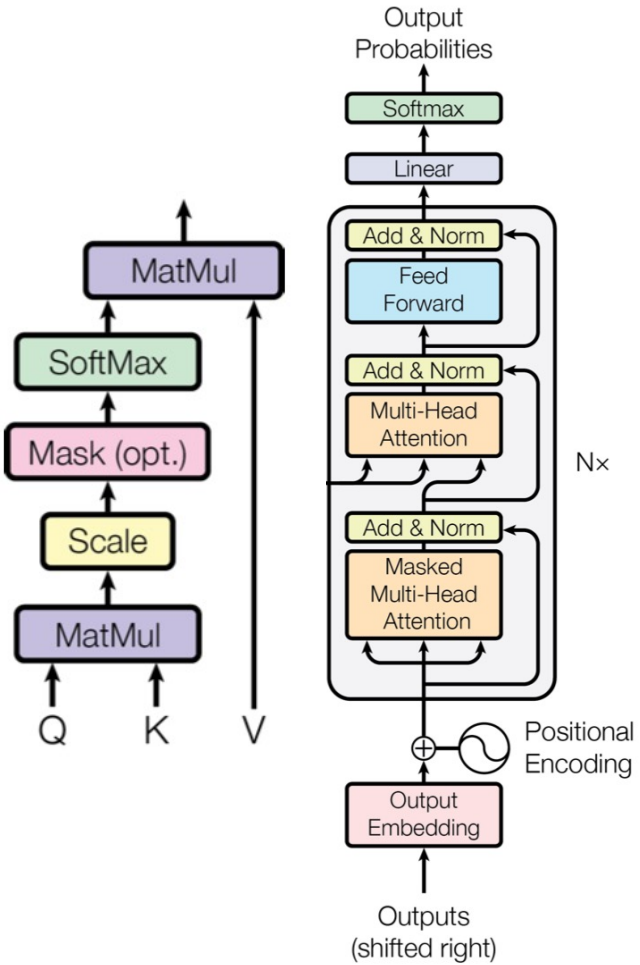
	<start>	Ich	habe	einen	Apfel	gegessen
<start>						
Ich						
habe						
einen					🚫	
Apfel						
gegessen						

Mask subsequent sequence elements. I.e., only allow to attend to positions up to and including the current position. This is achieved by setting softmax values for those to $-\infty$



	<start>	Ich	habe	einen	Apfel	gegessen
<start>		⊘	⊘	⊘	⊘	⊘
Ich			⊘	⊘	⊘	⊘
habe				⊘	⊘	⊘
einen					⊘	⊘
Apfel						⊘
gegessen						

Mask subsequent sequence elements. I.e., only allow to attend to positions up to and including the current position. This is achieved by setting softmax values for those to $-\infty$

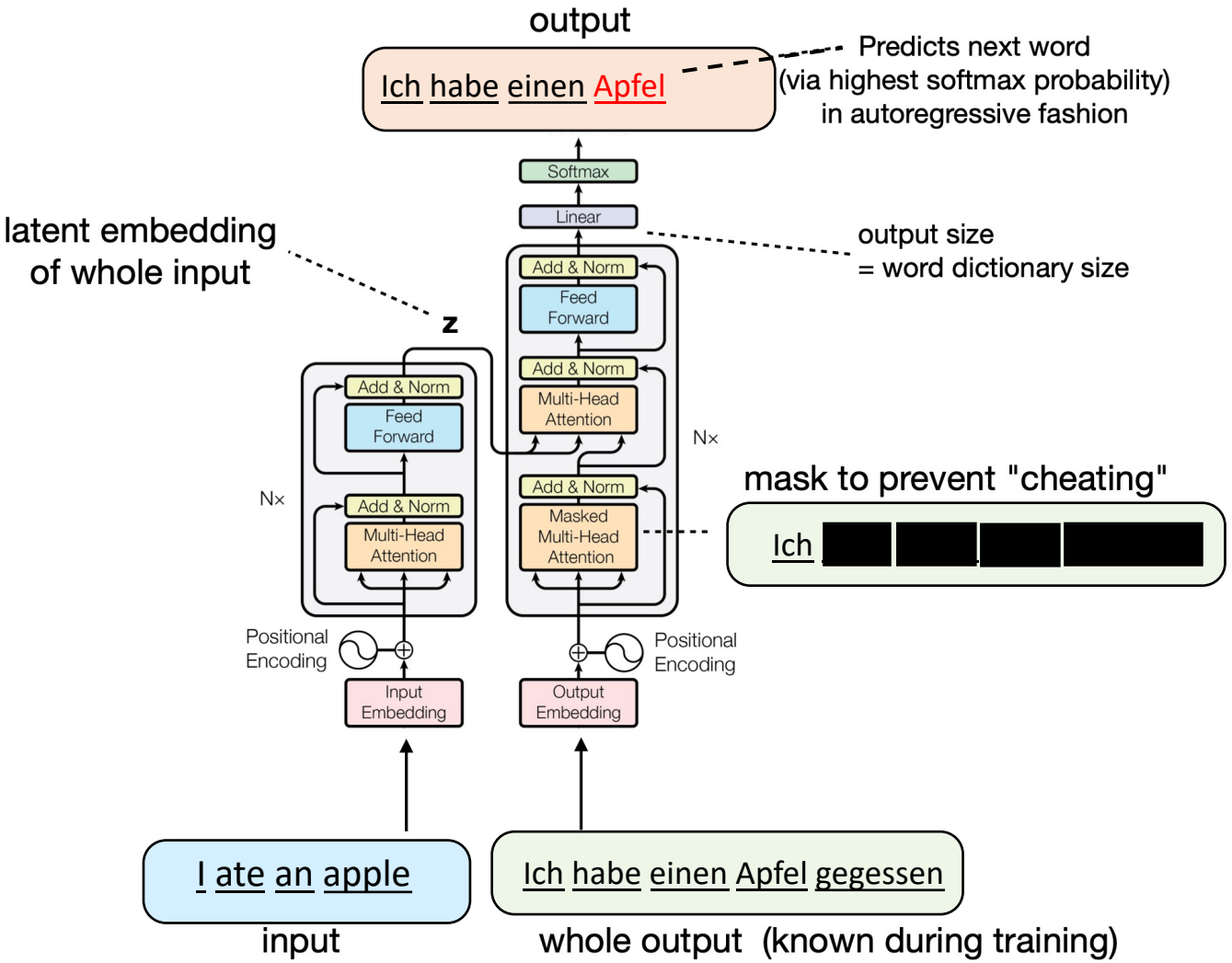


Scaled Score

Look-ahead Mask

0	-inf	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf	-inf
0	0	0	-inf	-inf	-inf
0	0	0	0	-inf	-inf
0	0	0	0	0	-inf
0	0	0	0	0	0

Mask subsequent sequence elements. I.e., only allow to attend to positions up to and including the current position. This is achieved by setting softmax values for those to $-\infty$


output
Ich habe einen Apfel
Predicts next word
(via highest softmax probability)
in autoregressive fashion
Softmax
Linear
latent embedding
of whole input
z
output size
= word dictionary size
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Add & Norm
Masked
Multi-Head
Attention
Nx
mask to prevent "cheating"
Ich
Add & Norm
Feed
Forward
Add & Norm
Multi-Head
Attention
Nx
Positional
Encoding
Positional
Encoding
Input
Embedding
Output
Embedding
I ate an apple
Ich habe einen Apfel gegessen
input
whole output (known during training)

Transformer

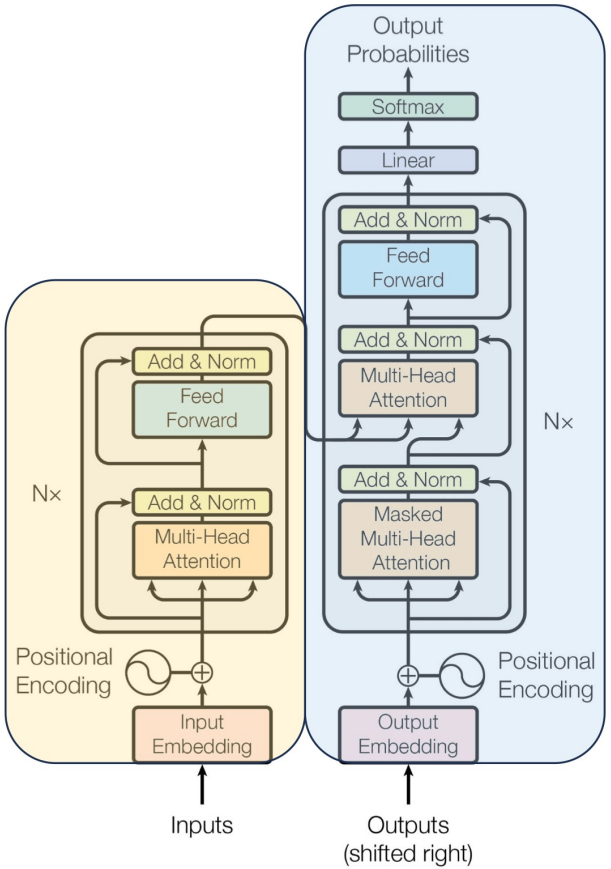
Input – input tokens
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Can also be adapted to generate tokens *by appending a module that maps hidden state dimensionality to vocab size*

Representation



Input – output tokens and hidden states*
Output – output tokens

Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

Can also be adapted to generate hidden states *by looking before token outputs*

Generation

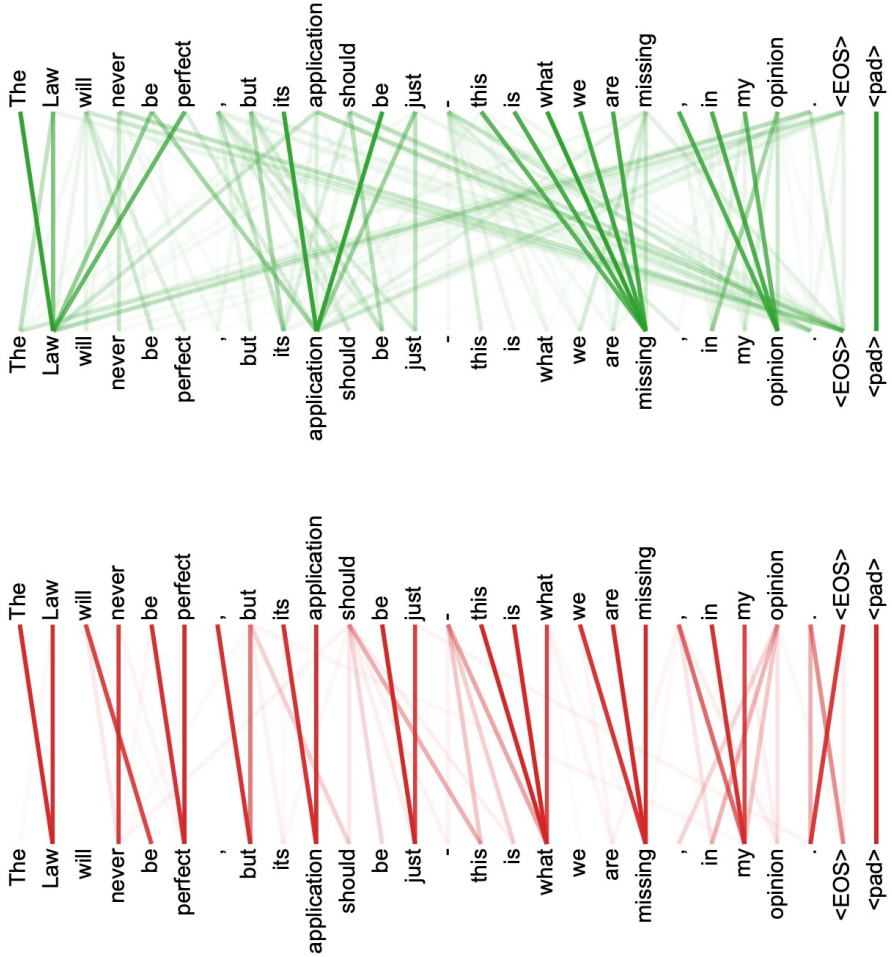
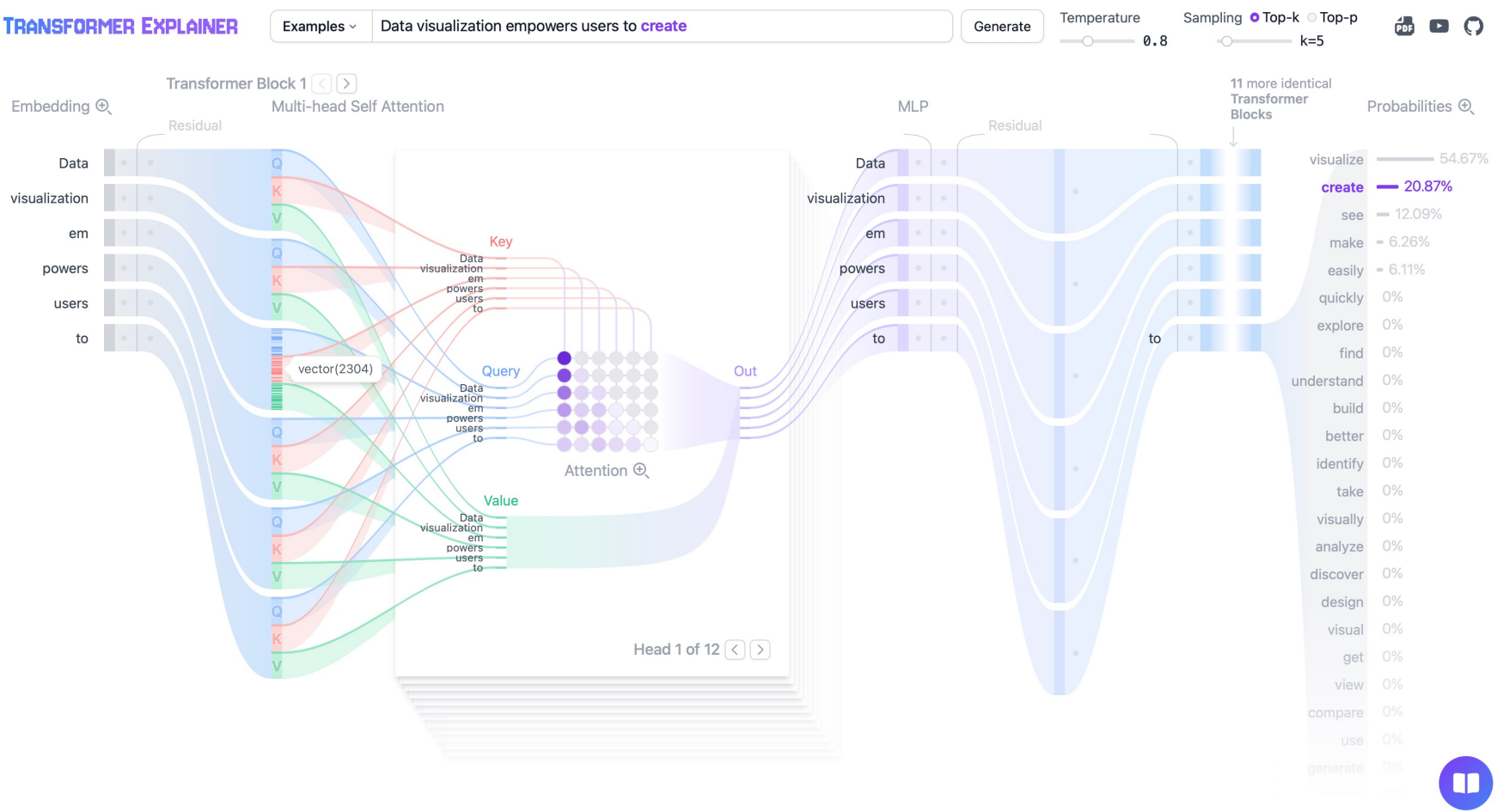
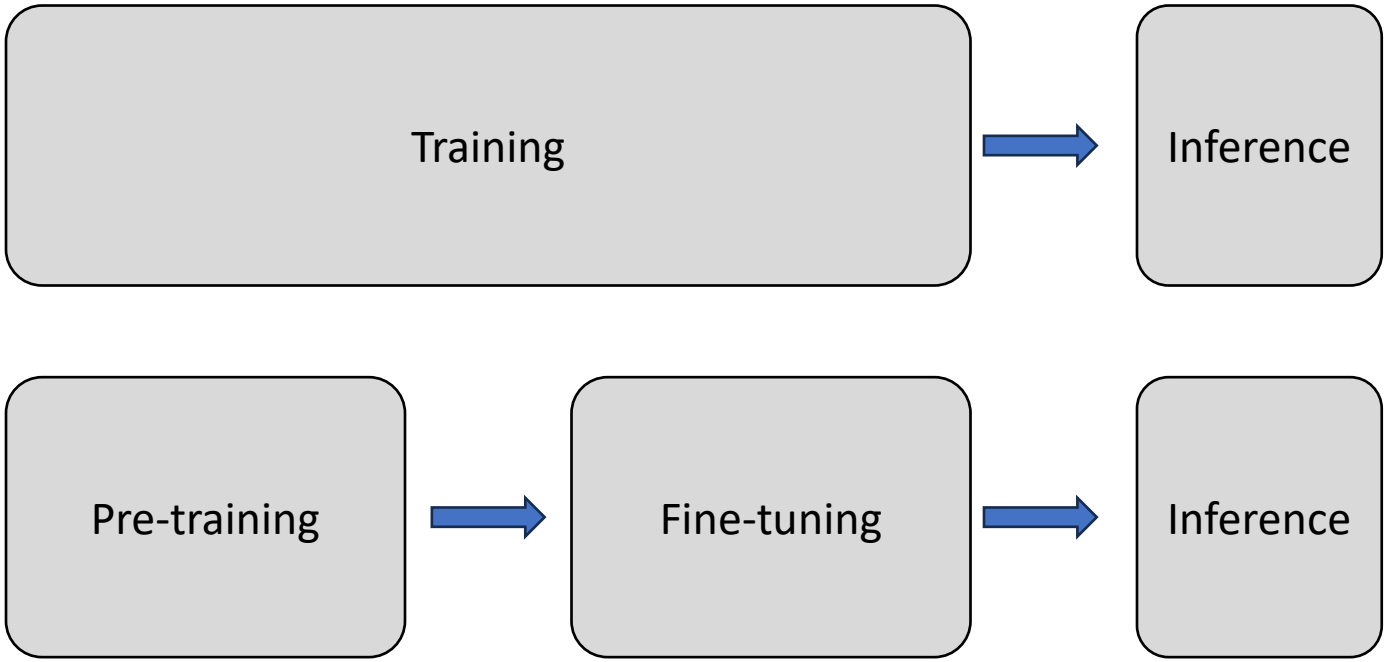


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

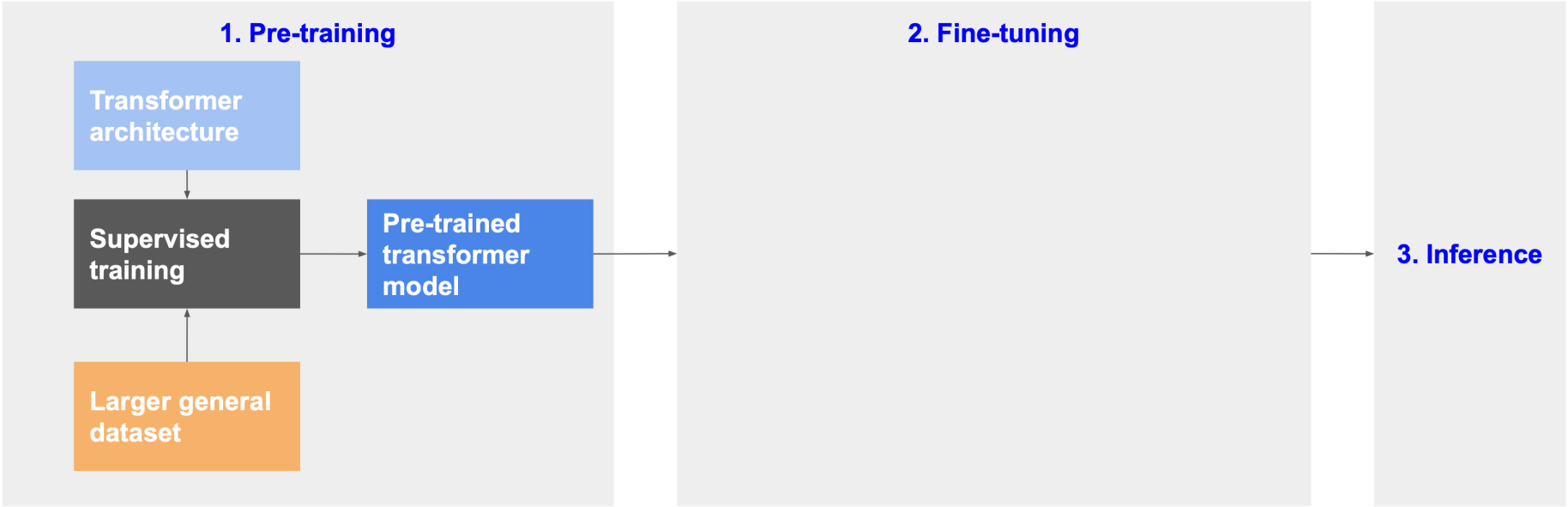


https://poloclub.github.io/transformer-explainer/

How to train and fine-tune transformers



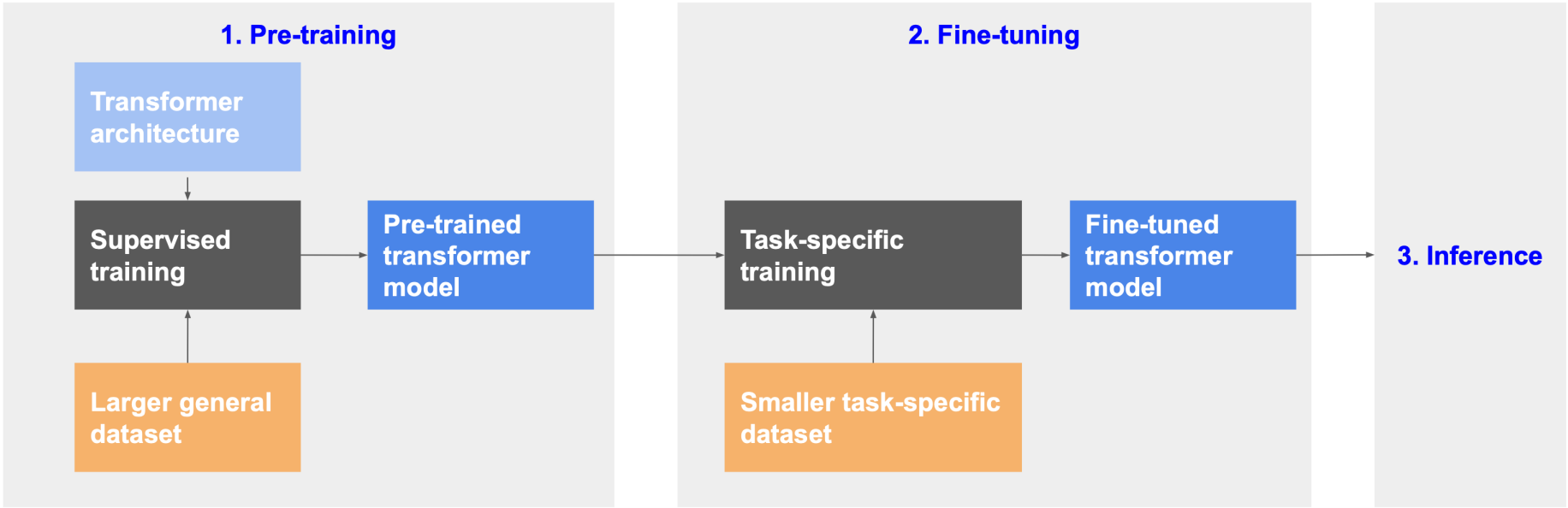
How to train and fine-tune transformers



Lot's of data, learn general things. May serve as a parameter initialization.

Usually requires significant computational resources and time.

How to train and fine-tune transformers



Lot's of data, learn general things. May serve as a parameter initialization.

Usually requires significant computational resources and time.

Adaptation to the specific task.

Potentially less computationally intensive.

Parameter-Efficient Fine-Tuning Techniques

LoRA (Lower-Rank Adaptation)

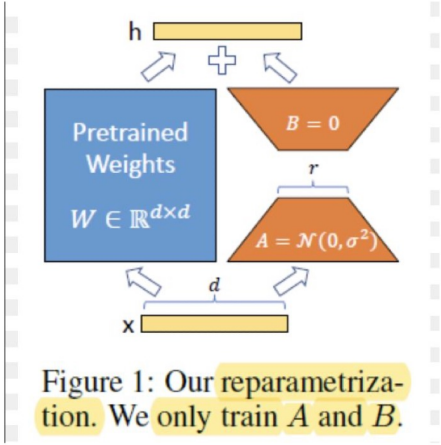
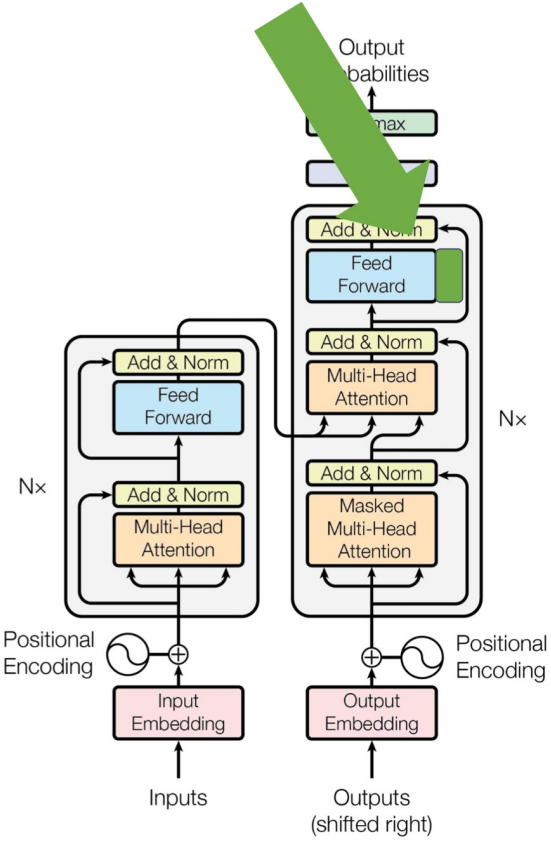


Figure 1: Our reparametrization. We only train A and B.

BitFit

$$\mathbf{Q}^{m,\ell}(\mathbf{x}) = \mathbf{W}_q^{m,\ell}\mathbf{x} + \mathbf{b}_q^{m,\ell}$$
$$\mathbf{K}^{m,\ell}(\mathbf{x}) = \mathbf{W}_k^{m,\ell}\mathbf{x} + \mathbf{b}_k^{m,\ell}$$
$$\mathbf{V}^{m,\ell}(\mathbf{x}) = \mathbf{W}_v^{m,\ell}\mathbf{x} + \mathbf{b}_v^{m,\ell}$$

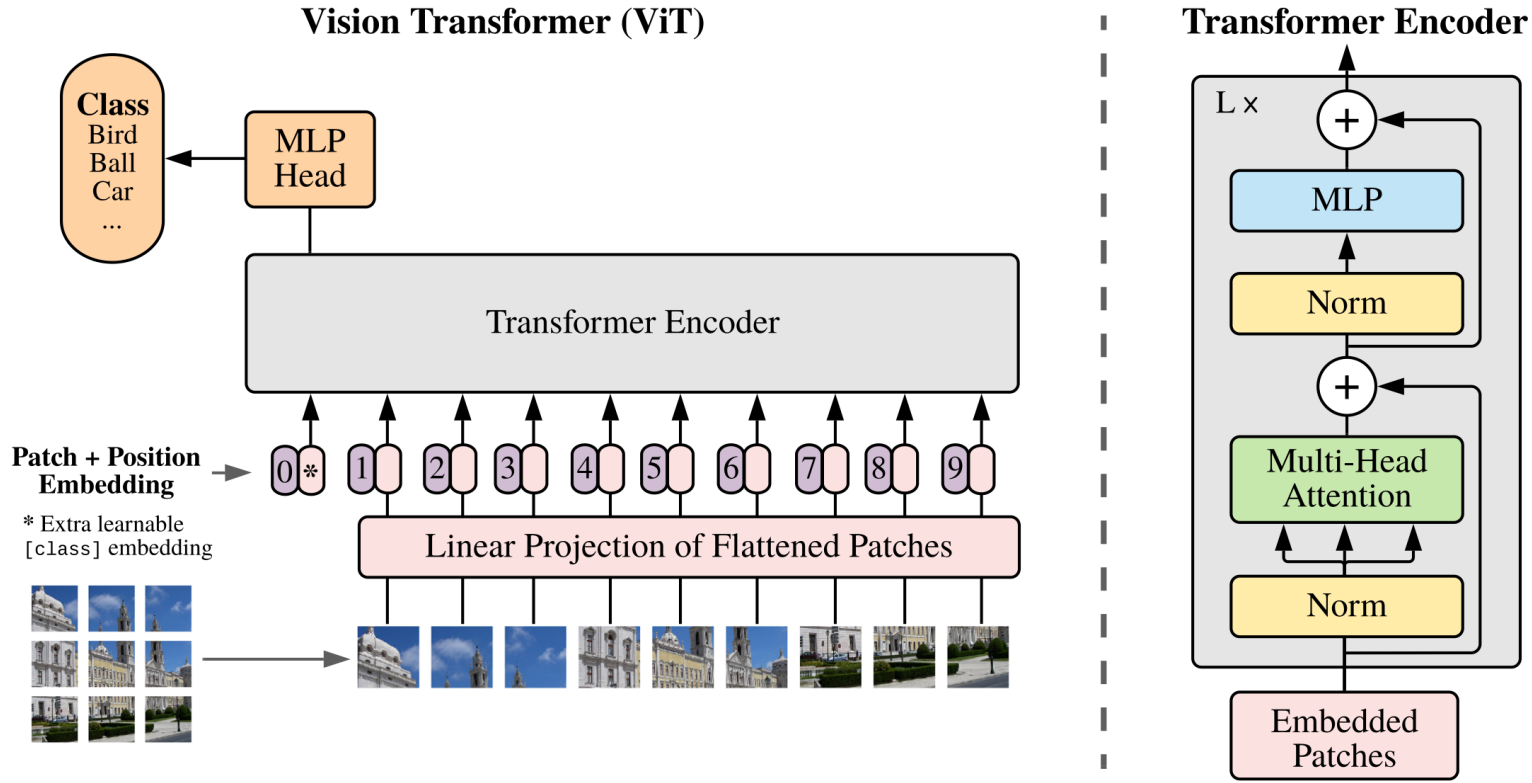


LoRA: https://arxiv.org/abs/2106.09685
BitFit: https://arxiv.org/abs/2106.10199

Vision transformer



Vision transformer



An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Vision transformer

Do vision transformers work well?

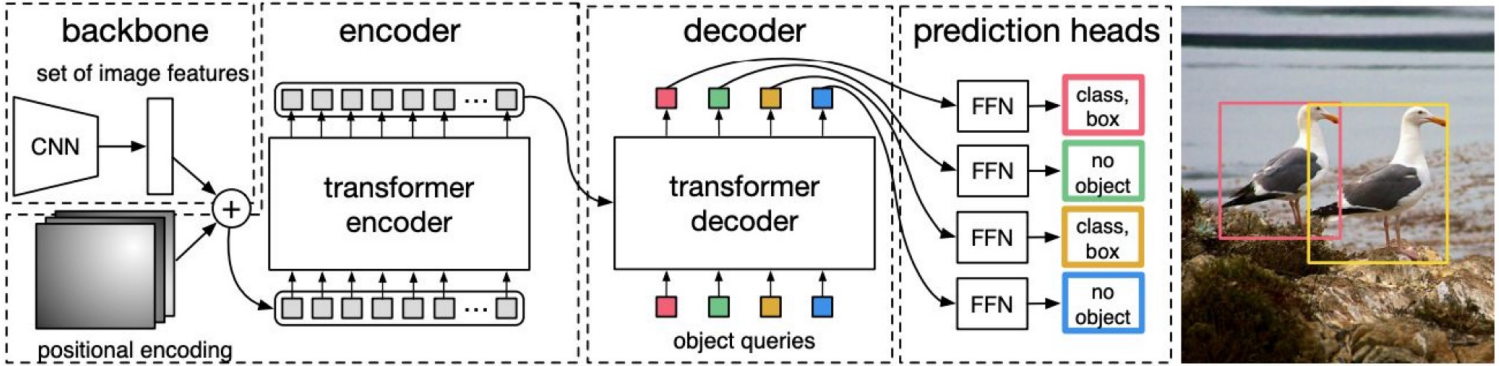
- ViT first Transformer-based vision model to achieve comparable results to state-of-the-art CNNs, while being more computationally efficient to train
- Transformer architecture has less inductive biases than CNNs (i.e., assumes less about the spatial structure than convolutional filter design does)
  - Consequence: Transformers works well when trained on very large amounts of data, less so when there are smaller / medium amounts of data (in this case, leveraging CNN's assumptions about data structure is helpful)
- Weakness: Transformer architectures such as ViT also memory-intensive
- Weakness: ViT cannot scale to high-resolution images (due to computational complexity of self-attention), and does not work for denser prediction tasks like object detection or segmentation

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Vision transformer

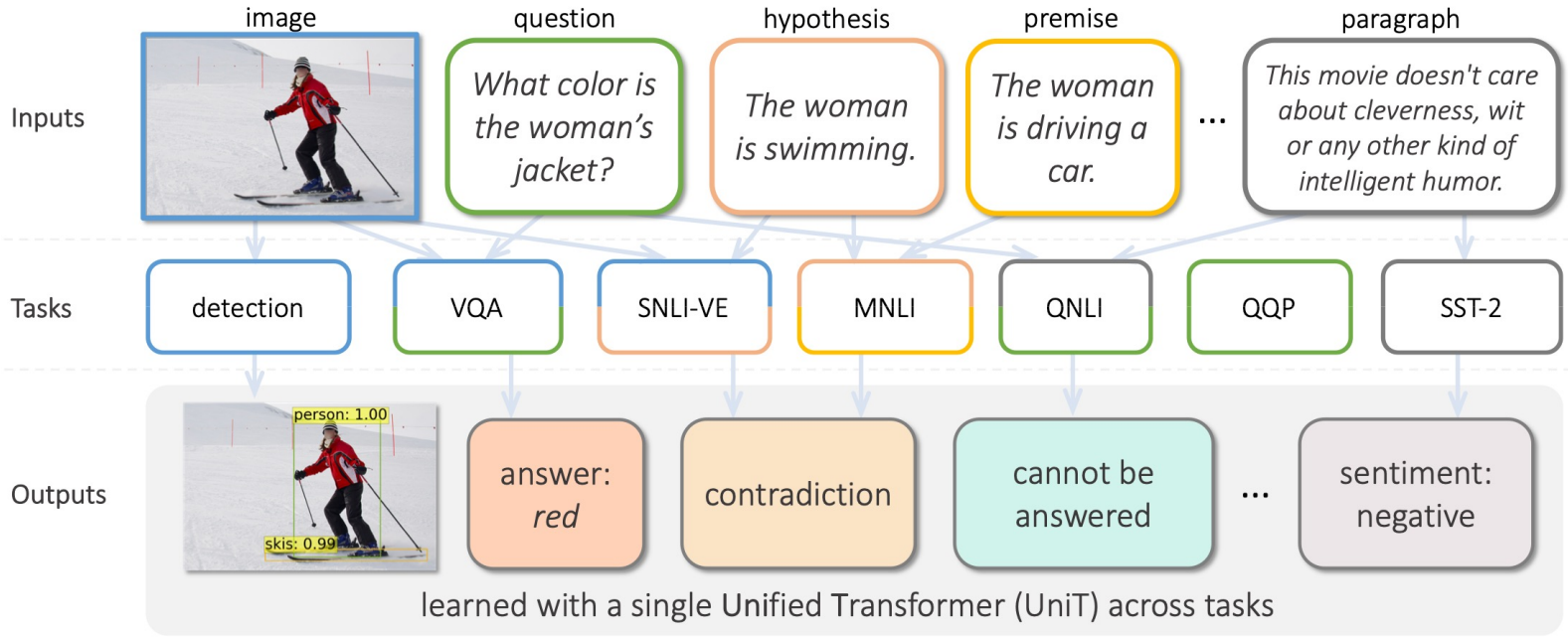
DETR: Transformer-based models for more complex vision tasks

- Uses a Transformer encoder-decoder model to perform detection and segmentation
- Trained directly end-to-end to predict all objects at once, with a set loss function that performs bipartite matching
- Allows avoiding previous hand-designed components of object detection models, like spatial anchors and non-maximal suppression!

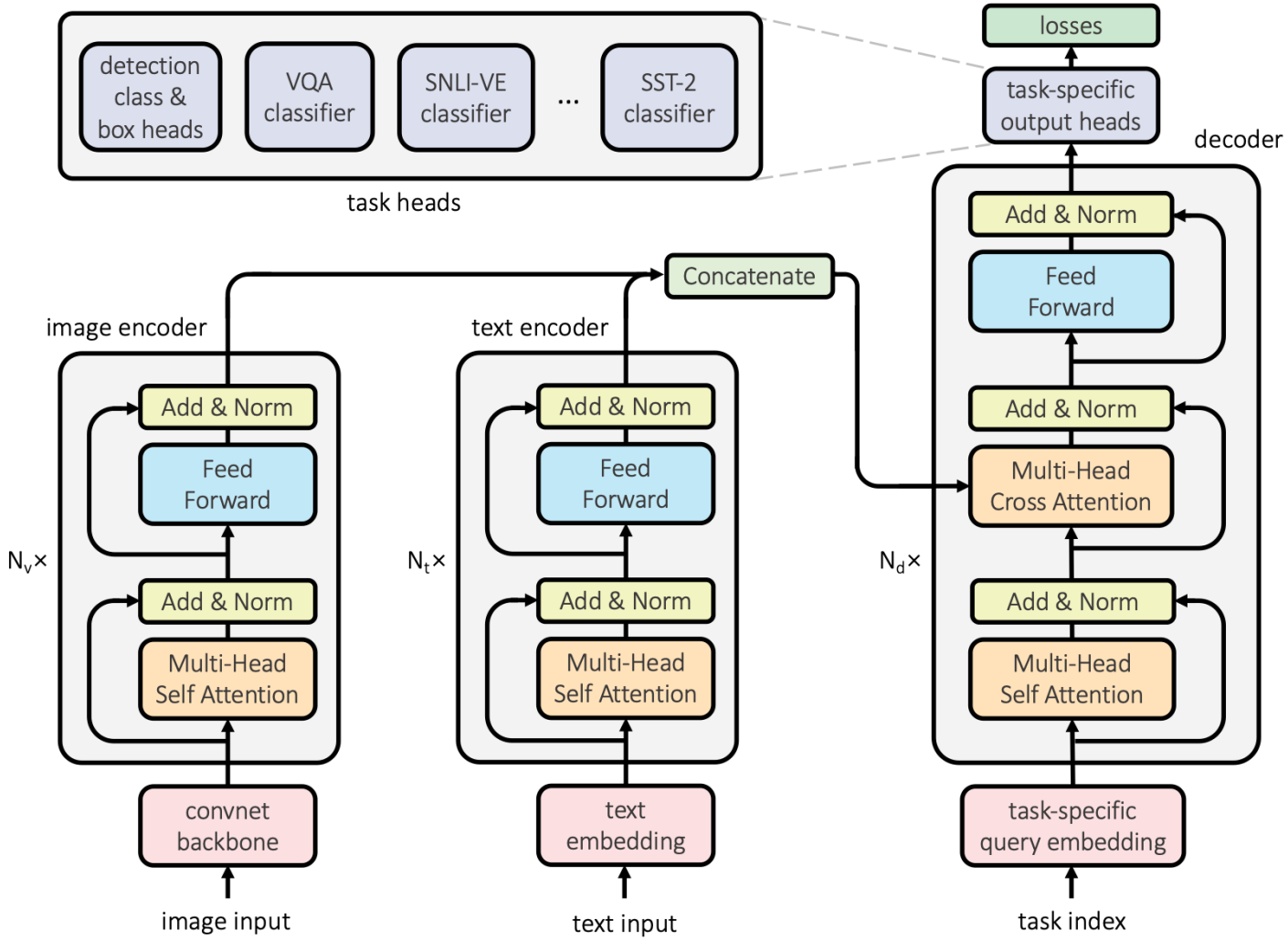


Carion et al. 2020

Multimodal transformer



UniT: Multimodal Multitask Learning with a Unified Transformer


losses
task-specific output heads
decoder
detection class & box heads
VQA classifier
SNLI-VE classifier
...
SST-2 classifier
task heads
Concatenate
image encoder
text encoder
Add & Norm
Feed Forward
Multi-Head Self Attention
Multi-Head Cross Attention
$N_v\times$
$N_t\times$
$N_d\times$
convnet backbone
text embedding
task-specific query embedding
image input
text input
task index

GPT (Generative Pre-trained Transformer)

GPT (110 million parameters)
Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

GPT-2 (1.5 billion parameters)
Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, *1*(8), 9.
https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

GPT-3 (175 billion parameters)
Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. arXiv preprint arXiv:2005.14165. https://arxiv.org/abs/2005.14165

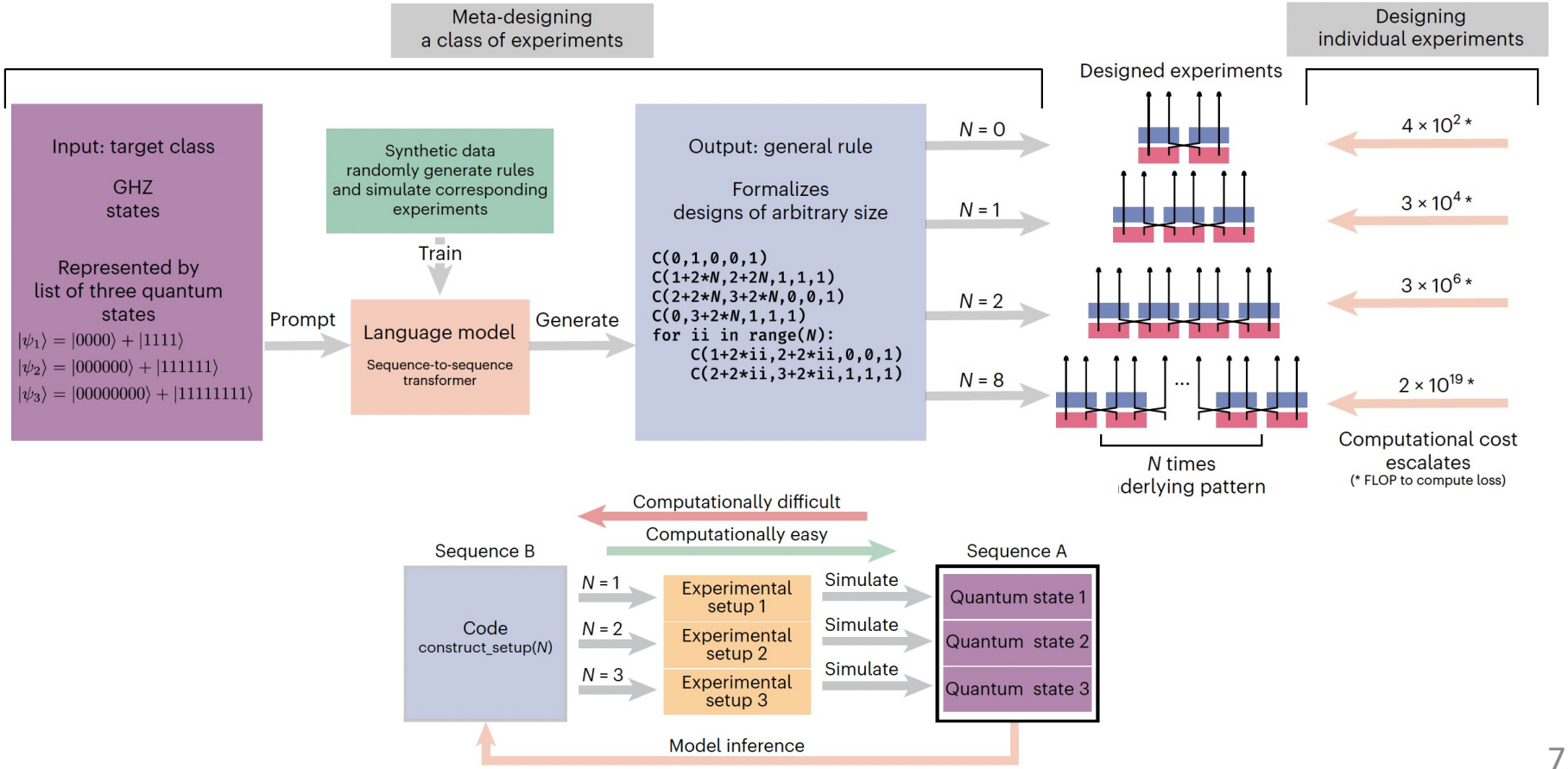
GPT-5: estimated to have **between 2 trillion and 5 trillion parameters**

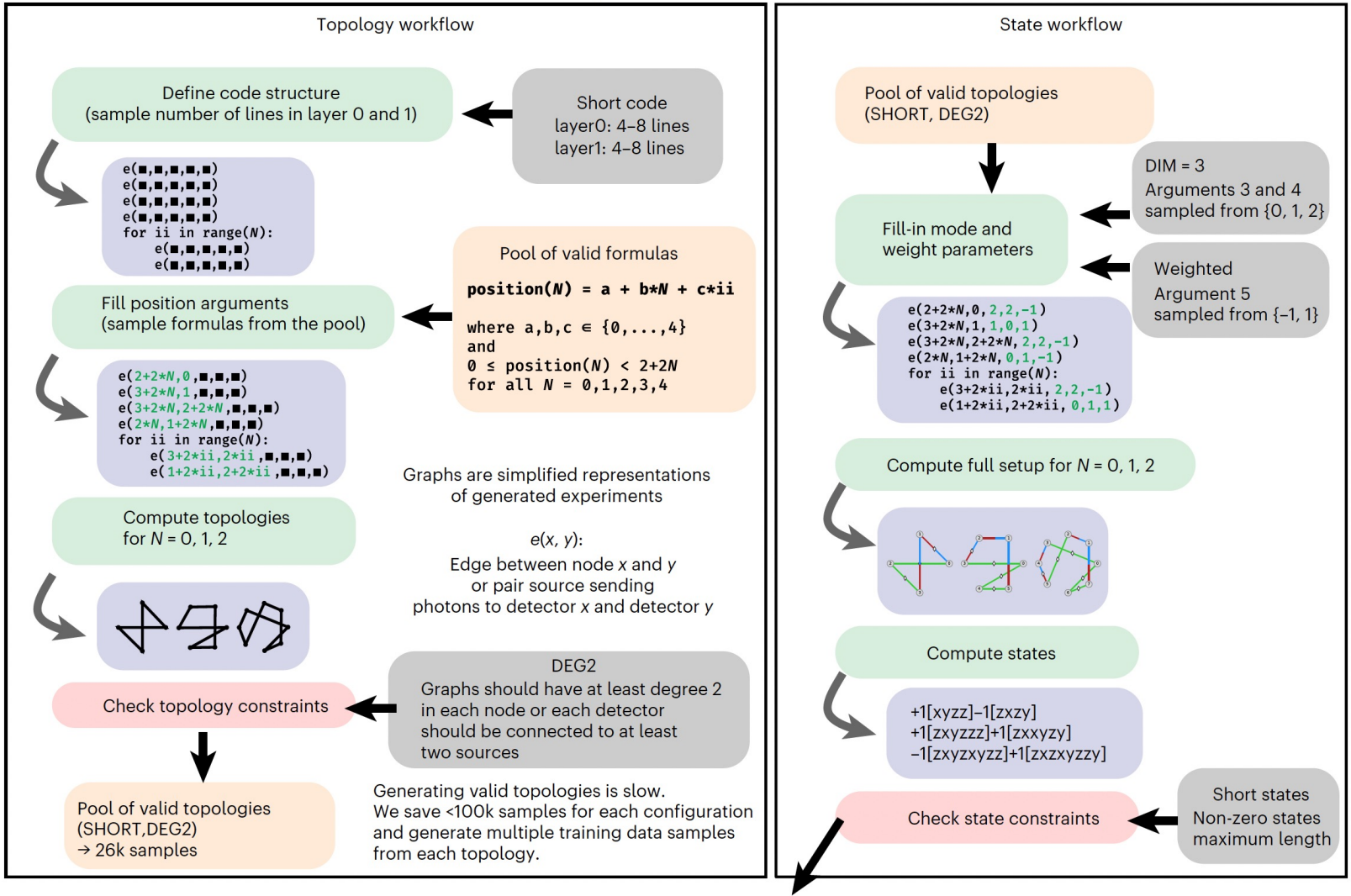
Some Examples in (Quantum) Physics

Meta-designing quantum experiments

https://arxiv.org/abs/2406.02470

How finding general design concepts can be automated.




Topology workflow
Define code structure
(sample number of lines in layer 0 and 1)
Short code
layer0: 4–8 lines
layer1: 4–8 lines
e(■,■,■,■,■)
e(■,■,■,■,■)
e(■,■,■,■,■)
e(■,■,■,■,■)
for ii in range(N):
e(■,■,■,■,■)
e(■,■,■,■,■)
Pool of valid formulas
position(N) = a + b*N + c*ii
where a,b,c ∈ {0,...,4}
and
0 ≤ position(N) < 2+2N
for all N = 0,1,2,3,4
Fill position arguments
(sample formulas from the pool)
e(2+2*N,0,■,■,■)
e(3+2*N,1,■,■,■)
e(3+2*N,2+2*N,■,■,■)
e(2*N,1+2*N,■,■,■)
for ii in range(N):
e(3+2*ii,2*ii,■,■,■)
e(1+2*ii,2+2*ii,■,■,■)
Graphs are simplified representations
of generated experiments
e(x, y):
Edge between node x and y
or pair source sending
photons to detector x and detector y
Compute topologies
for N = 0, 1, 2
DEG2
Graphs should have at least degree 2
in each node or each detector
should be connected to at least
two sources
Check topology constraints
Pool of valid topologies
(SHORT,DEG2)
→ 26k samples
Generating valid topologies is slow.
We save <100k samples for each configuration
and generate multiple training data samples
from each topology.
State workflow
Pool of valid topologies
(SHORT, DEG2)
DIM = 3
Arguments 3 and 4
sampled from {0, 1, 2}
Fill-in mode and
weight parameters
Weighted
Argument 5
sampled from {–1, 1}
e(2+2*N,0,2,2,-1)
e(3+2*N,1,1,0,1)
e(3+2*N,2+2*N,2,2,-1)
e(2*N,1+2*N,0,1,-1)
for ii in range(N):
e(3+2*ii,2*ii,2,2,-1)
e(1+2*ii,2+2*ii,0,1,1)
Compute full setup for N = 0, 1, 2
Compute states
+1[xyzz]–1[zxzy]
+1[zxyzzz]+1[zxxyzy]
–1[zxyzxyzz]+1[zxzxyzzy]
Check state constraints
Short states
Non-zero states
maximum length

Training data sample
(SHORT CODE, DEG2, DIM3, WEIGHTED, SHORT STATE)

Sequence A:

<SOS>+1[axbyczdz]-1[azbxczdy]|+1[azbxcydzezfz]+1[azbxcxdyezfy]|-1[azbxcydzexfygzhz]+ 1[azbxczdxeyfzgzhy]<EOS>

Sequence B:

<SOS>e(2+2*N,0,2,2,-1)\ne(3+2*N,1,1,0,1)\ne(3+2*N,2+2*N,2,2,-1)\ne(2*N,1+2*N,0,1,-1)\nfor ii in range(N):\n e(3+2*ii,2*ii,2,2,-1)\n e(1+2*ii,2+2*ii,0,1,1)<EOS>

Tokenization vocabulary

```
"<PAD>": 0,   "0": 3,    "ax": 13,   "ay": 21,   "az": 29,   "[": 37,    "for ii in range(N):": 43,
"<SOS>": 1,   "1": 4,    "bx": 14,   "by": 22,   "bz": 30,   "]": 38,    "\n": 44,
"<EOS>": 2,   "2": 5,    "cx": 15,   "cy": 23,   "cz": 31,   "|": 39,    "  ": 45,
              "3": 6,    "dx": 16,   "dy": 24,   "dz": 32,   "+": 40,    "e(": 46,
              "4": 7,    "ex": 17,   "ey": 25,   "ez": 33,   "-": 41,    ")": 47,
              "5": 8,    "fx": 18,   "fy": 26,   "fz": 34,   "*": 42,    ",": 48,
              "6": 9,    "gx": 19,   "gy": 27,   "gz": 35,               "N": 49,
              "7": 10,   "hx": 20,   "hy": 28,   "hz": 36,               "ii": 50
              "8": 11,
              "9": 12,
```

Tokenized training data sample

Sequence A:

```
[1 40  4 37 13 22 31 32 38 41  4 37 29 14 31 24 38 39 40  4 37 29 14 23
32 33 34 38 40  4 37 29 14 15 24 33 26 38 39 41  4 37 29 14 23 32 17 26
35 36 38 40  4 37 29 14 31 16 25 34 35 28 38  2]
```

Sequence B:

```
[1 44 46  5 40  5 42 49 48  3 48  5 48  5 48 41  4 47 44 46  6 40  5 42
 49 48  4 48  4 48  3 48  4 47 44 46  6 40  5 42 49 48  5 40  5 42 49 48
  5 48  5 48 41  4 47 44 46  5 42 49 48  4 40  5 42 49 48  3 48  4 48 41
  4 47 44 43 44 45 46  6 40  5 42 50 48  5 42 50 48  5 48  5 48 41  4 47
 44 45 46  4 40  5 42 50 48  5 40  5 42 50 48  3 48  4 48  4 47 44  2]
```

SHORT CODE
DEG1
DIM2
UNWEIGHTED
SHORT STATES

SHORT CODE
DEG1
DIM2
UNWEIGHTED
LONG STATES

•••

SHORT CODE
DEG2
DIM3
WEIGHTED
SHORT STATES

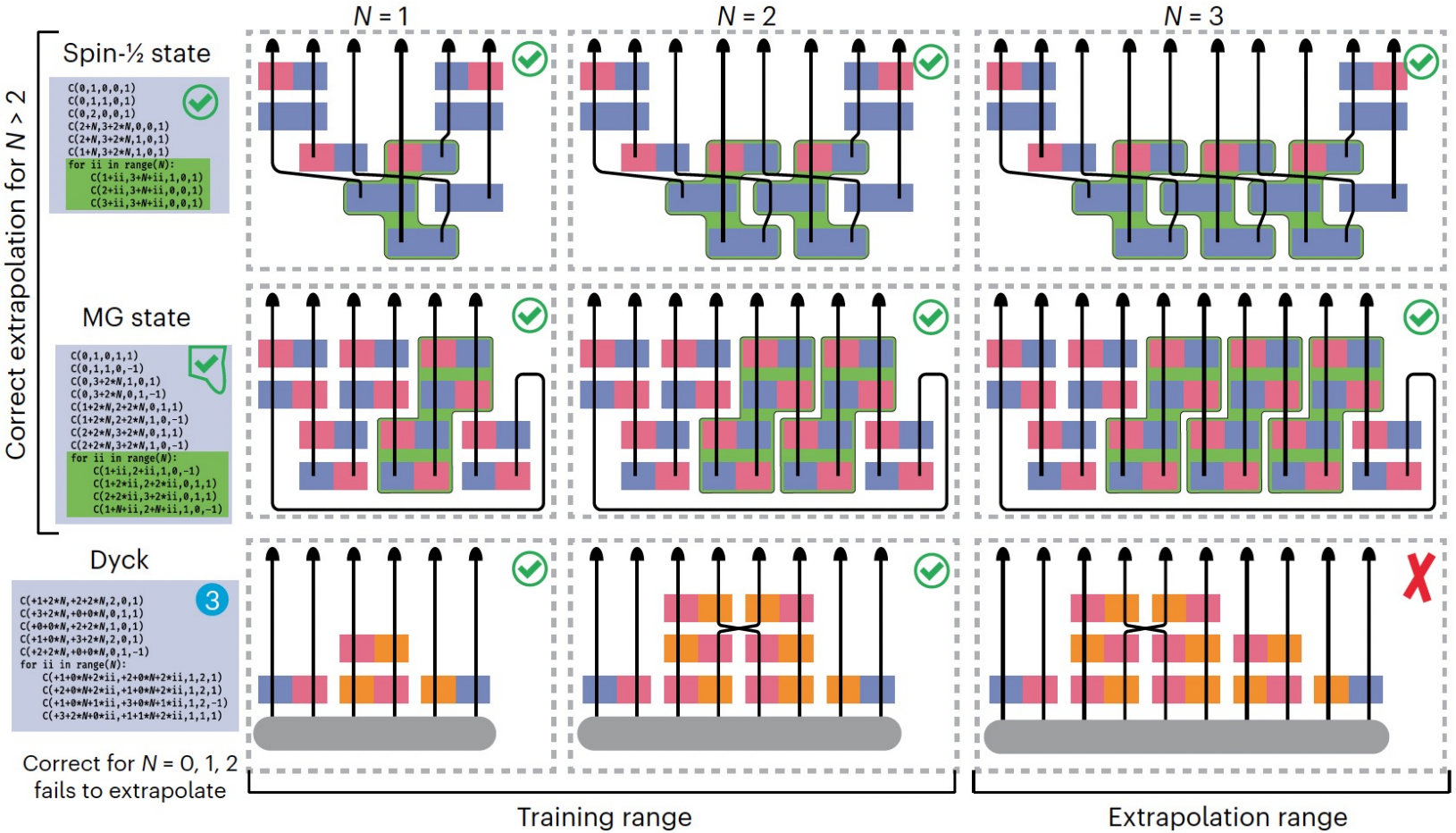
•••

LONG CODE
DEG2
DIM3
WEIGHTED
SHORT STATES

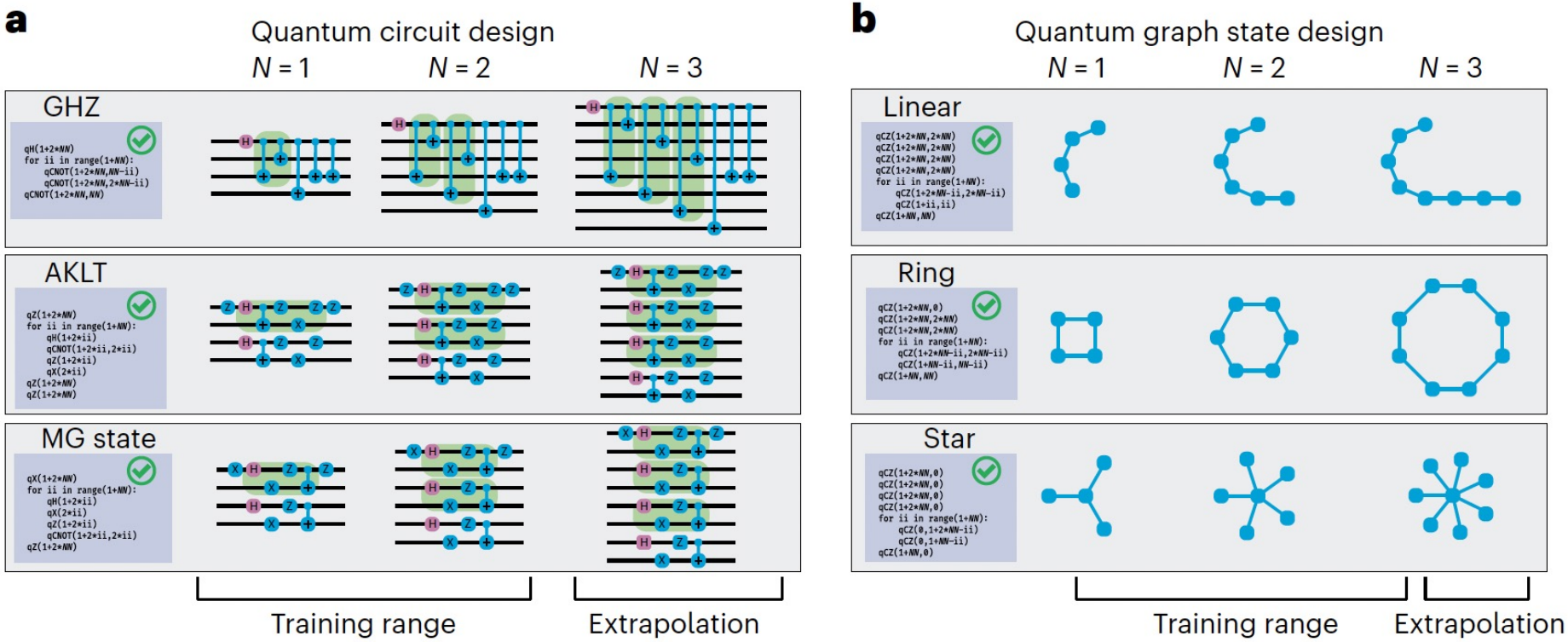
LONG CODE
DEG2
DIM3
WEIGHTED
LONG STATES

Combined training data (56 million samples)

Meta-designing quantum experiments

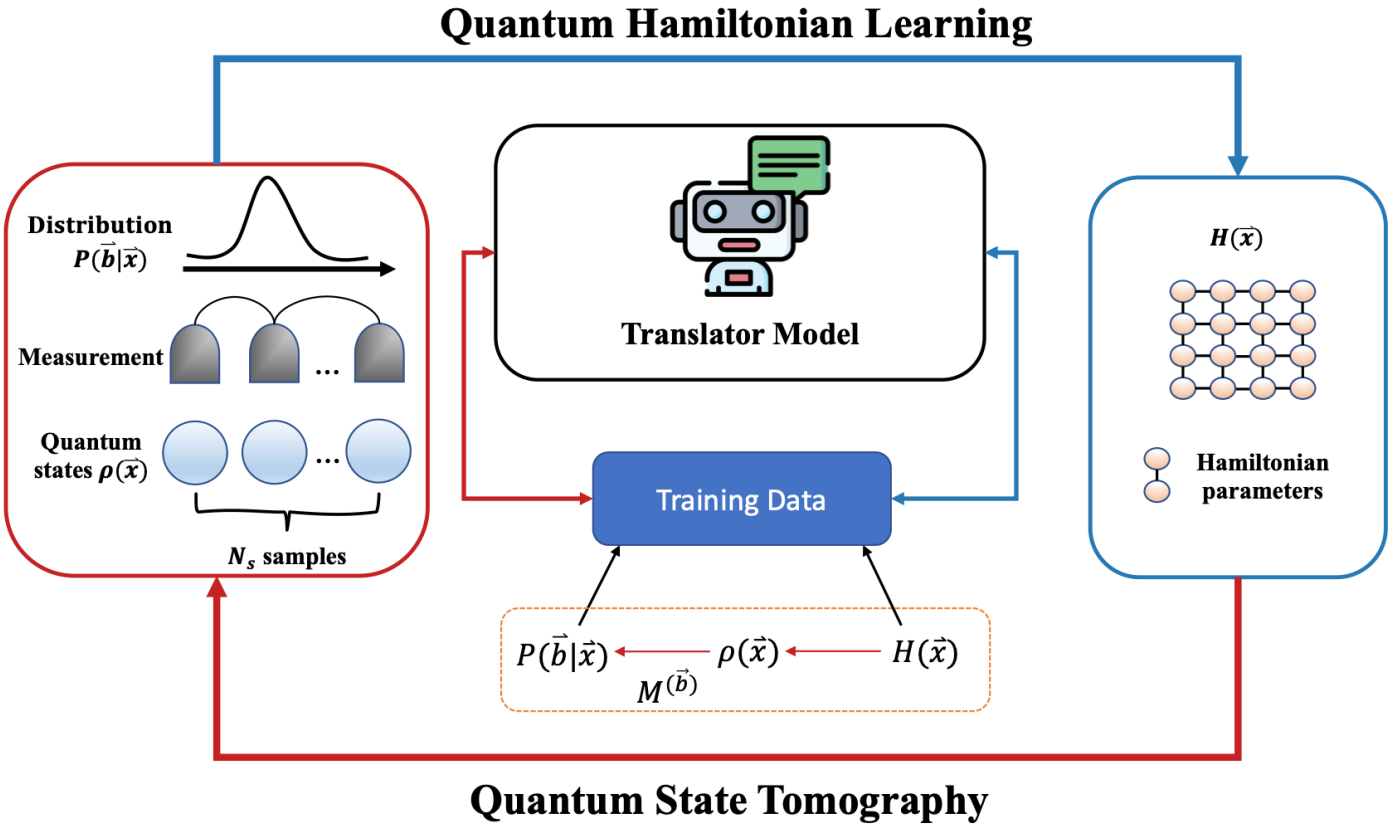


Meta-designing quantum experiments



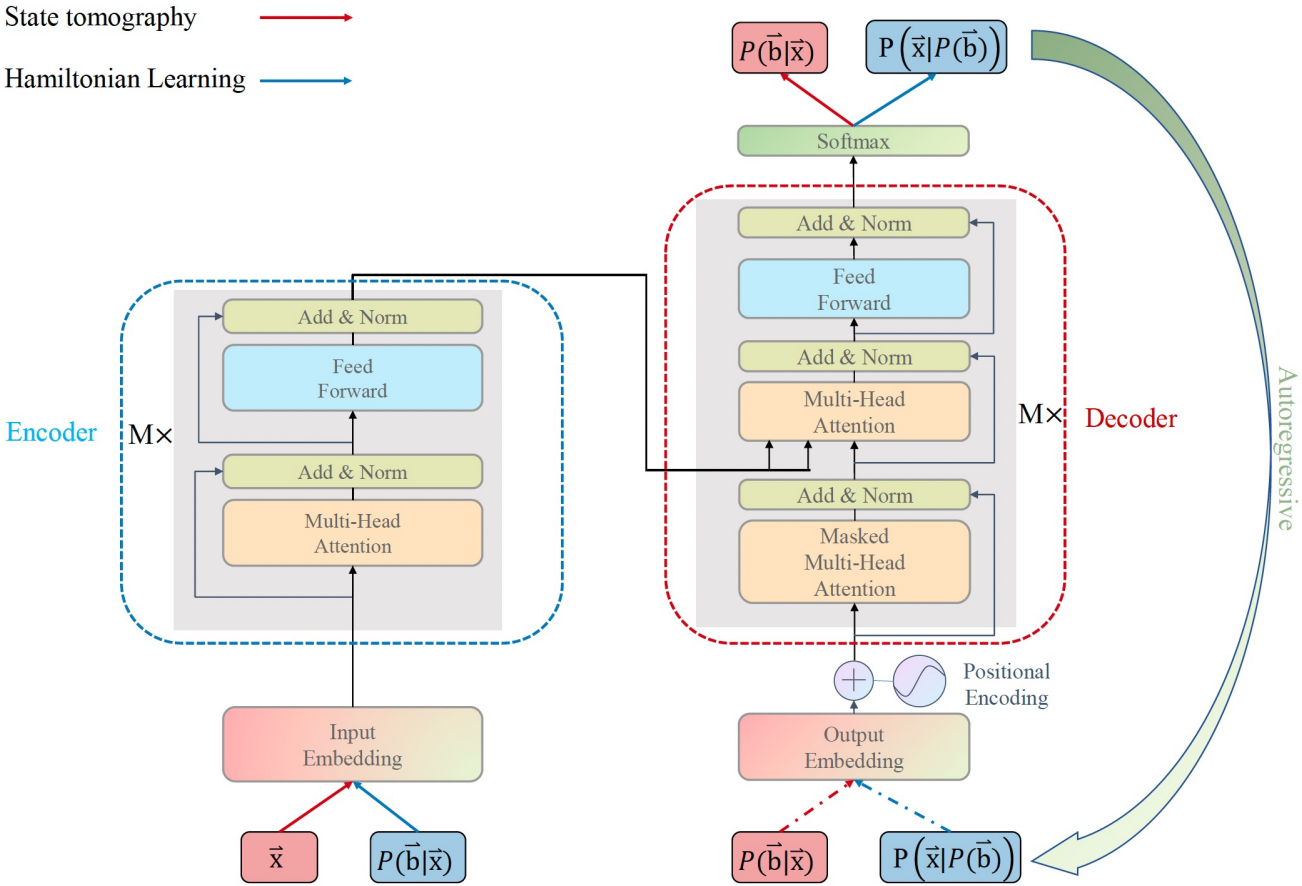
Quantum State Tomography and Hamiltonian Learning

https://doi.org/10.1103/PhysRevApplied.21.014037

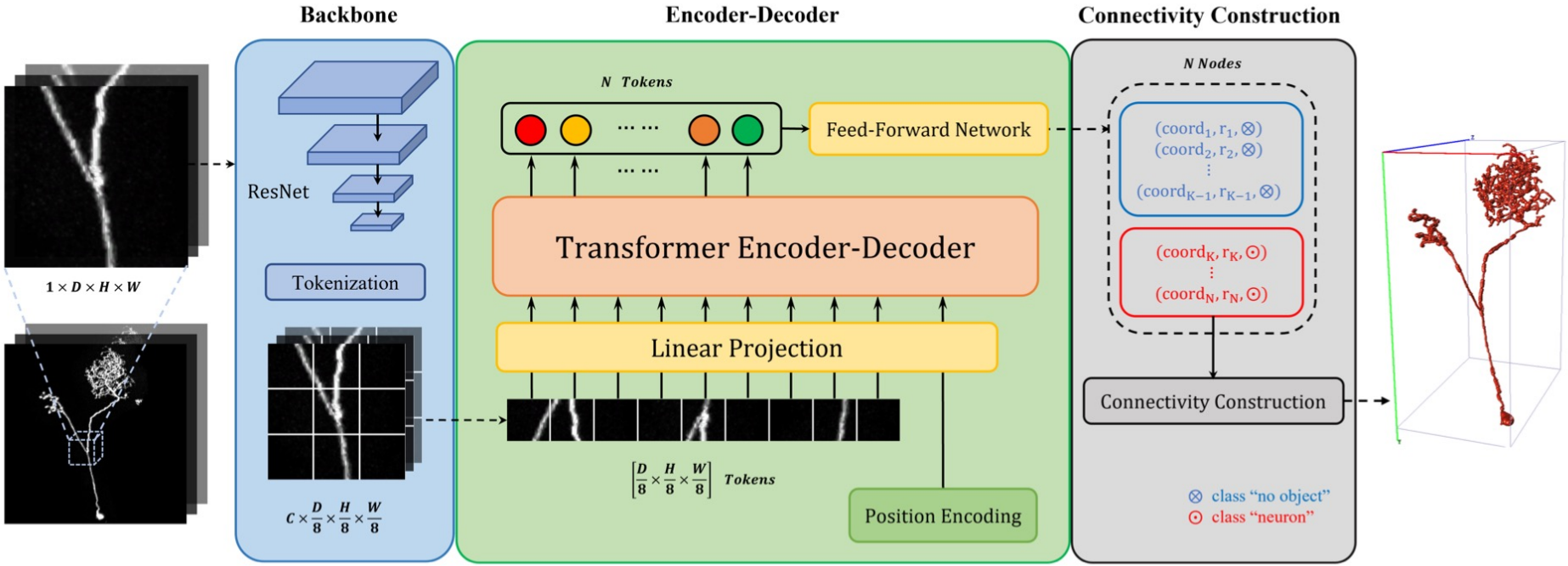


Quantum State Tomography and Hamiltonian Learning

https://doi.org/10.1103/PhysRevApplied.21.014037



3D neuron morphology



https://arxiv.org/abs/2212.04163